Aras Product Engineering 14

Administrator Guide

Document #: ***D-008157***

Last Modified: *8/22/2024*

Copyright Information

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Phone: 978-691-8900

E-mail: support@aras.com

Website: https://www.aras.com/

Table of Contents

Send Us Your Comments

Aras Corporation welcomes your comments and suggestions on the quality and usefulness of this document. Your input is an important part of the information used for future revisions.

- Did you find any errors?
- Is the information clearly presented?
- Do you need more information? If so, where and what level of detail?
- Are the examples correct? Do you need more examples?
- What features did you like most?

If you find any errors or have any other suggestions for improvement, indicate the document title, and the chapter, section, and page number (if available).

You can send comments to us in the following ways:

Email:
TechDocs@aras.com
Subject: Aras Product Documentation

Or,

Postal service:
Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Attention: Aras Technical Documentation

If you would like a reply, provide your name, email address, address, and telephone number.

If you have usage issues with the software, visit https://www.aras.com/support

1 Overview

The Aras Product Engineering (PE) application is a business-ready solution for product definition and engineering change management across the enterprise. It includes core PLM data and process functionality such as Parts, Bill of Materials (BOM), Documents, CAD Documents, Approved Manufacturer Lists / Approved Vendor Lists (AML/AVL) and Change items.

The Aras Product Engineering application includes the following benefits:

- Visibility into key development indicators increases engineering effectiveness and provides a basis for continuous improvement.
- Secure online management of product definition processes fosters the development of high-quality products.
- Engineering change management reduces cycle times and streamlines business operations.
- Controlled product information releases between departments, divisions, suppliers, and customers provides effective coordination.
- Reuse of product and process information increases productivity and controls costs.
- Industry best practices for configuration management and engineering change are incorporated using CMII principles and processes.
- Unparalleled flexibility enables application modification to address proprietary engineering processes and specific competitive practices.

The Administrator Guide describes the options and procedures available for configuring and customizing the Product Engineering application.

2 Configuring Identities

Setting up Identities in the Aras Product Engineering application is similar to building a company organization chart. The configuration process is as follows:

- Create a Group Identity for each top-level structural unit (department). For example, a company may have the following departments: Engineering, Sales, Finance, and Marketing.
- For each top-level Identity, create a hierarchy of Group Identities reflecting the structure. For example, Engineering may have Development, Support, and QA. Support may further consist of Customer Support, Documentation, and Training.
- Create Teams for Express Change Management.
- Create individual User Identities.
- Assign each of the User Identities to the appropriate Group Identities.
- Assign Identities involved in change management to Change Management Group Identities and Teams.
- Assign Identities responsible for managing Effectivity on Bill of Materials (BOM) to the Effectivity Management Identity.

2.1 Understanding Change Management Roles

The CMII, Simple Change Management, and Express Change Management processes are available in the Product Engineering application. Each of these change processes has its own Change Management ItemTypes and Workflows. They are discussed in detail in the *Aras Product Engineering 14 - User Guide*.

Each Change Management ItemType is handled by specific Roles in its Workflow. These Roles are represented as Change Management (CM) Identities, Change Management (CM) Teams, and System Identities:

- The CM Identities are used in the CMII and Simple Change Management Workflows.
- The CM Teams, as well as the CM Identities, are used in the Express Change Management Workflows.
- The System Identities are Roles that the system defines, such as an Item Creator or Owner.

As an Administrator, you must assign User and Group Identities to the CM Identities and Teams used in your workflows.

Warning Each required Role must be populated with the necessary Identities before a Change Management Workflow Process starts.

2.2 Change Management Identities

Aras Product Engineering has built-in Group Identities that hold specific permissions: control Workflows of specific Items, Item creation, Item access, etc. These Identities and their Roles are based on the CMII standards. These Identities are:

- **CM group—the Configuration Management group**—consists of CSI, CSII, and CSIII:
  - **CSI—Change Specialist I**—the person who drives the change process in a company. The CSI determines who is technically responsible for problem verification, which changes have higher priority than others, whether an ECR (Engineering Change Request) can be approved through a fast track or if it needs review board involvement. The CSI is also responsible for determining if the required change has a wide impact and can bring others into the review process.
  - **CSII—Change Specialist II**—the person who completes the change process in a company. The CSII decides when approved changes get implemented, when actual items get changed, and if any requests can be combined into a single change notice.
  - **CSIII—Change Specialist III**—the person who implements the change in the production process of an affected item. The CSIII decides when the change goes into production and how it affects the existing production flow. By default, CSIII is active in a Change Management Workflow during the last phases of an ECN (Engineering Change Notice).
- **CRB—the Change Review Board**—the people responsible for several steps in the ECR process. When an ECR has wide or complex implications, the CRB is engaged to review the change and provide input into the process.
- **Component Engineering**—the people responsible for managing the Parts that a company uses as well as the sourcing aspects of these Parts: Manufacturers, Vendors, and the Lifecycles associated with these Items.
- **All Employees**—the employees of the company.
- **All Customers**—the customers of the company.
- **All Suppliers**—the suppliers that a company uses.

2.3 Change Management Teams

To access or create a Team, click the Navigation icon  to access the Table of Contents (TOC) and click **Administration → Teams**. You can search for existing teams or create new ones. The following figure shows the Product Team form.

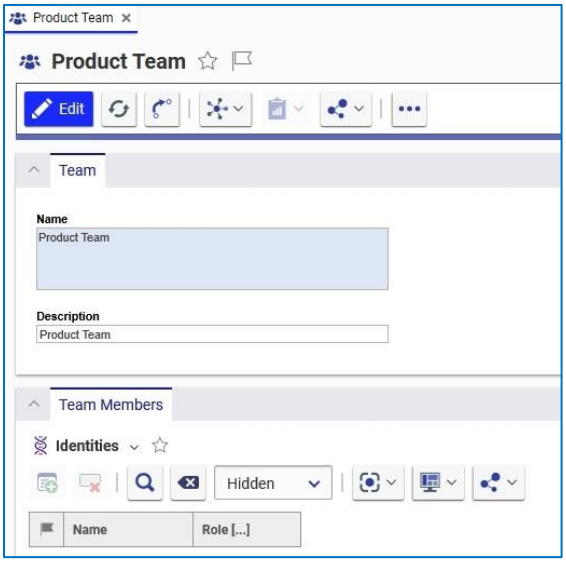

Figure 1.

The **Product Team** is used by default in the Express ECO and Express DCO Change Items.

As an Administrator, you must populate the Teams with all the Team Roles necessary for the Express Change Items used at your company:

- **Team Manager**—plans and reviews the changes.
- **Team Member**—reviews and implements the changes. Some Workflows require several Team Members, e.g., one assigns a Review Task to another.
- **Team Guest**—is used for notifying a specific Identity not involved in the Workflow.
- **Team**—is used when it is necessary to notify the whole Team.

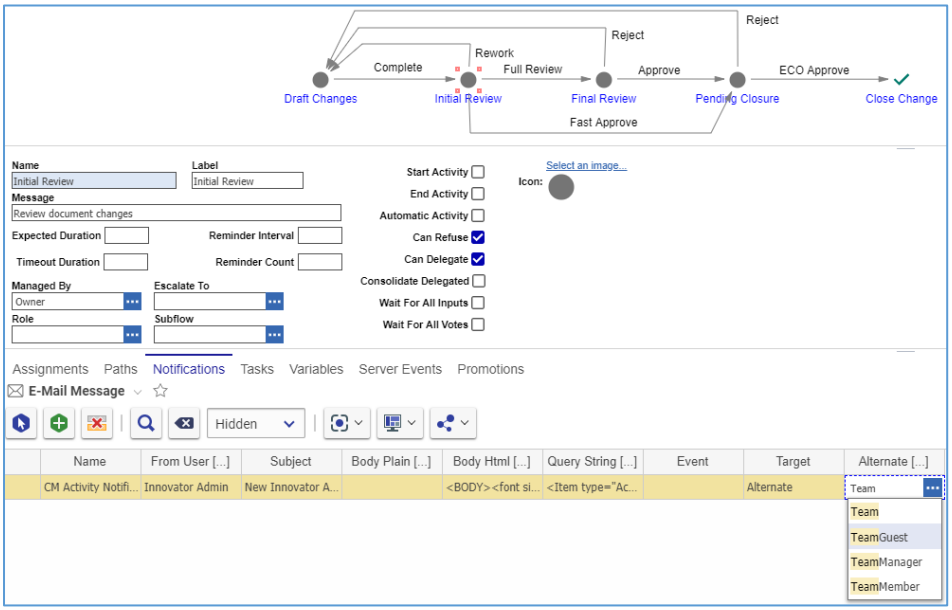

Figure 2.

Within a Team, one Identity can have several Roles and several Identities can have the same Role.

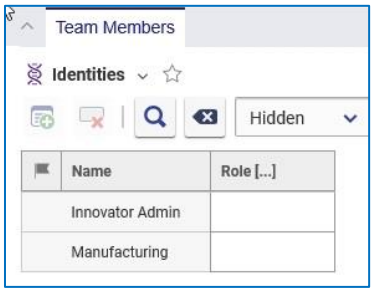

Figure 3.

2.3.1 Changing the Default Team on Express Change Items

When you create an Express Change Item, the **Team** property is prepopulated with **Product Team** by default.

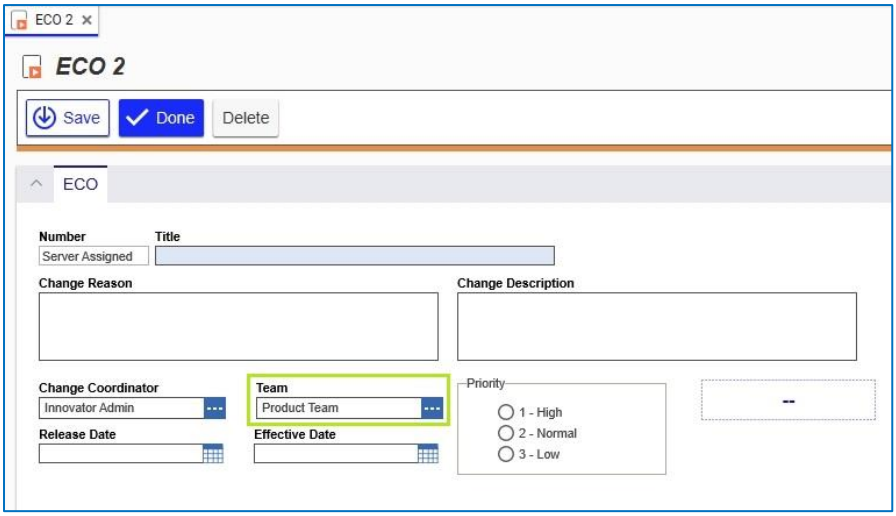

Figure 4.

The **Client Events** accordion tab of each ***Express Change ItemType*** Item view includes the **PE_SetTeam** Method, which defines the default Team. The **OnAfterNew** Event invokes the method.

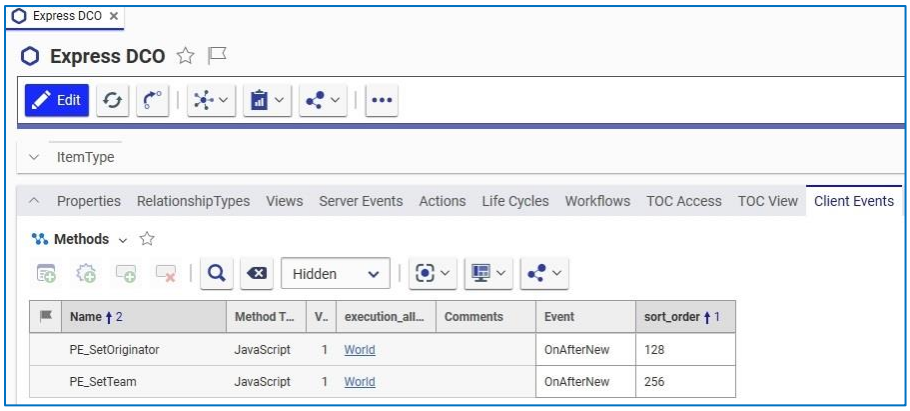

Figure 5.

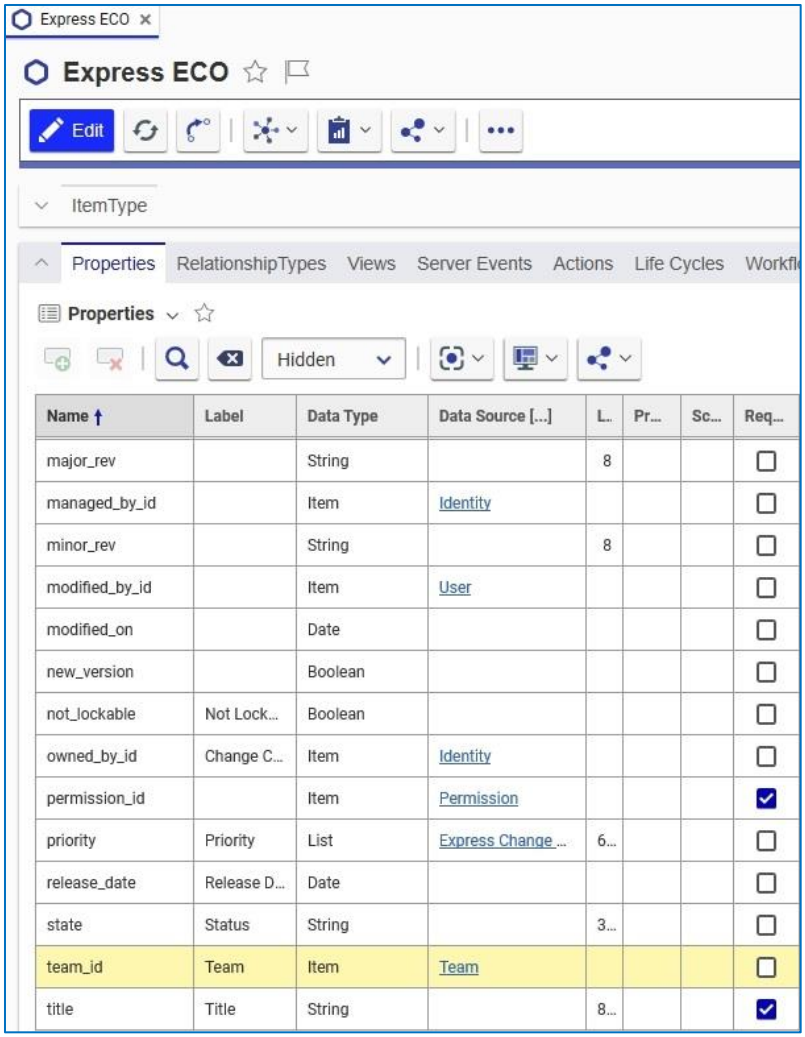

Figure 6.

Note: **PE_SetTeam** is a system Method and should not be deleted or modified. However, you can change the second argument of `team.setProperty()` to specify another Team as the default per your business process.

2.4 System-Defined Identities

The system defines the following Identities:

- **Aras PLM**—a system Role representing system processes. It does not contain any individuals.
- **Manager**—an individual assigned to the **Designated User** property on the Item view. This Identity is responsible for the management, usage, and review of the Item.
- **Owner**—an individual assigned to the **Assigned Creator** property on the Item view. This Identity is the Item owner.
- **Creator**—an individual assigned to the **Created By** property on the Item. This is the Identity that creates the Item. This property is assigned automatically and cannot be changed.

Warning The **Aras PLM** Identity permissions must not be deleted or changed.

Users specify the **Manager** or **Owner** of Items by populating the corresponding Item properties. By default, these properties are not required on Item views. If **Manager** or **Owner** is used in Workflow or business processes, you can make the corresponding property required to ensure users will not miss this requirement.

2.5 Effectivity Management Identity

The **Effectivity Management** Identity is responsible for managing Effectivity Scopes, Effectivity Variables, and Effectivity on Bill of Materials (BOMs).

By default, the **Effectivity Management** Identity includes the **Administrators** Identity. You can add other User and Group Identities.

3 Configuring Permissions

This section describes the out-of-the-box Permissions used in the Product Engineering (PE) application. Item Permissions define the Access Rights that Identities have for performing actions on an Item.

From the **Permissions** accordion tab on an ItemType form, you can define permissions for the ItemType. The **Is Default** check box indicates that a particular Permission is the default upon item's creation.

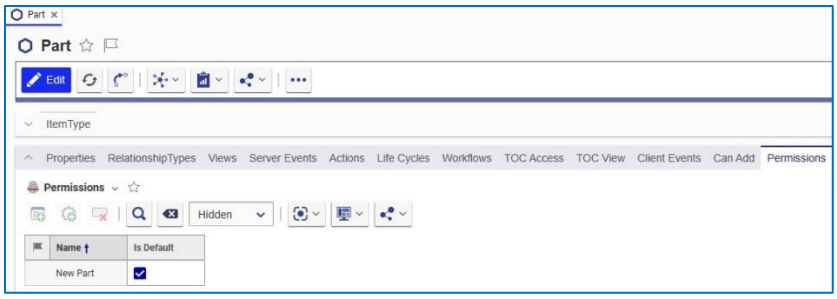

Figure 7.

Items may be associated with Life Cycle Maps. The Life Cycle Maps can change the Item Permissions at a particular Life Cycle state.

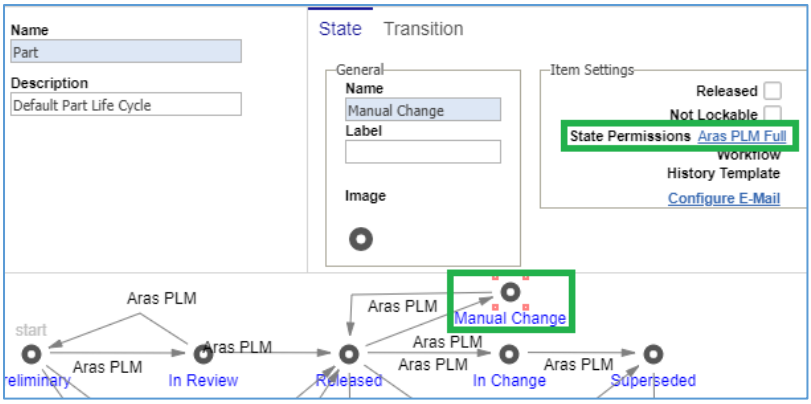

Figure 8.

From the **Can Add** accordion tab on an ItemType form, you can configure the create Permissions.

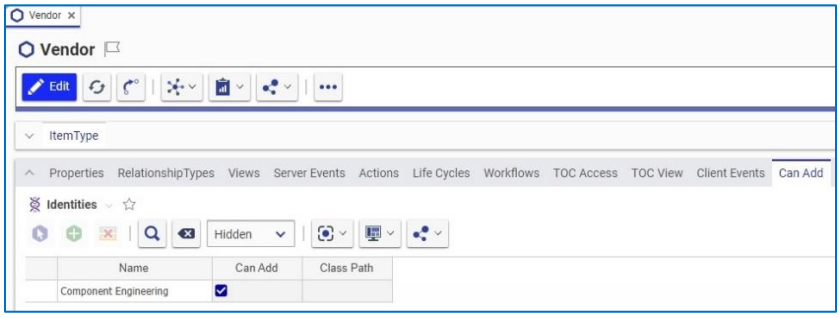

Figure 9.

3.1 Design Permissions

The following table describes the out-of-the-box Permissions for the **Part**, **Document**, and **CAD Document** ItemTypes. The **Life Cycle State** column lists the life cycle states. The Add, Get, Update, Delete, and Change access columns list the associated Identities which are allowed those Permissions for each Life Cycle State.

The **Promoted by** column shows the Identities that can promote an Item to a given Life Cycle State. If this column shows **Aras PLM**, an internal system process promotes the Item automatically.

The **Defining Permissions** column refers to Permissions that manage Access Rights for a particular Life Cycle State.

Table 1: Design permissions matrix

Life Cycle State	Add	Get	Update	Delete	Change access	Promoted by	Defining Permissions
Preliminary	All Employees, World—Documents	All Employees	Manager, Owner, Creator, Aras PLM	Owner, Creator, Aras PLM	Owner, Creator, Aras PLM	Starting State	New Part New Document New CAD
In Review		All Employees	Aras PLM	Aras PLM	Aras PLM	Aras PLM	In Review Part In Review Document In Review CAD
Released		All Employees	Aras PLM	Aras PLM	Aras PLM	Aras PLM, Owner	Released Part Released Document Released CAD
In Change		All Employees	Aras PLM	Aras PLM	Aras PLM	Aras PLM	Released Part Released Document Released CAD
Manual Change		All Employees	Aras PLM	Aras PLM	Aras PLM	Aras PLM	Aras PLM Full
Obsolete		All Employees	Aras PLM	Aras PLM	Aras PLM	Aras PLM	Released Part Released Document Released CAD
Superseded		All Employees	Aras PLM	Aras PLM	Aras PLM	Aras PLM	Released Part Released Document Released CAD

3.2 Change Management Permissions

Each of the Change Management ItemTypes has its own Permissions. Refer to the corresponding subsections of the *Change Management* section in the *Aras Product Engineering 14 - User Guide* for detailed descriptions of the Change Processes and Permissions.

3.3 Effectivity Access Control

There are two levels of access control for Effectivity. The **Effectivity Management** Permission defines the standard Access Rights to Effectivity. The **effs_releasedPartPolicy** Mandatory Access Control (MAC) Policy provides additional restrictions in addition to the **Effectivity Management** Permission.

3.3.1 Effectivity Management Permission

The **Effectivity Management** Permission provides Access Rights to Effectivity in Bill of Material (BOM) structures, Effectivity Scopes, Effectivity Variables, and [Effectivity] Models. The out-of-the-box Permission definition has the following Identities:

- **Effectivity Management** can view, create, update, and delete Effectivity in BOM structures as well as Items of the Effectivity Scope, Effectivity Variable, and Model ItemTypes. This Identity can resolve BOM structures by Effectivity Criteria.
- **World** can view Effectivity on BOM structures and resolve BOM structures using Effectivity Criteria.

3.3.2 MAC Policy to Control Effectivity Changes on Released Part BOMs

Out of the box, the **Effectivity Management** Identity can change Effectivity on BOM structures whose parent Parts are released. If necessary, changing Effectivity on released BOMs can be prevented by activating the **effs_releasedPartPolicy** Mandatory Access Control (MAC) Policy.

Refer to *Aras Innovator - MAC Policies* for information about activating and deactivating MAC policies.

3.4 Sourcing Permissions

Each of the **Manufacturer**, **Manufacturer Part**, and **Vendor** ItemTypes has the same-name default Permission. All these Permissions have the same Access Rights definition. The **Manufacturer** and **Vendor** Life Cycle Maps do not override these Permissions.

4 Configuring Item Number Sequences

A Sequence is an Item of the **Sequence** ItemType that defines a template for automatic sequential numbering of Items of the same ItemType.

Out of the box, each Change Management ItemType is provided with a default Sequence configuration. It is possible to change the default configuration and/or configure other ItemTypes to use Sequences.

When creating an Item, the property with the Sequence configuration is not editable and contains the **Server Assigned** text.

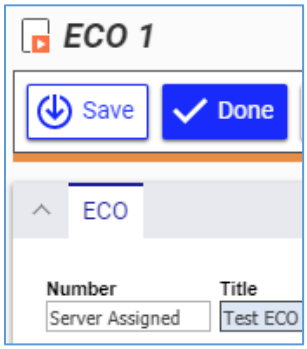

Figure 10.

Once the Item is saved, the property gets the next sequential number according to the defined template.

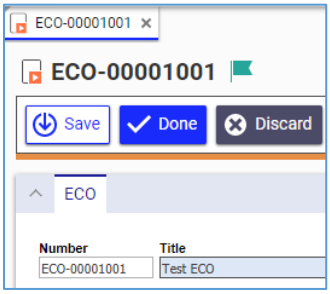

Figure 11.

The following figure shows the Sequence configuration for the Express ECO ItemType, as an example.

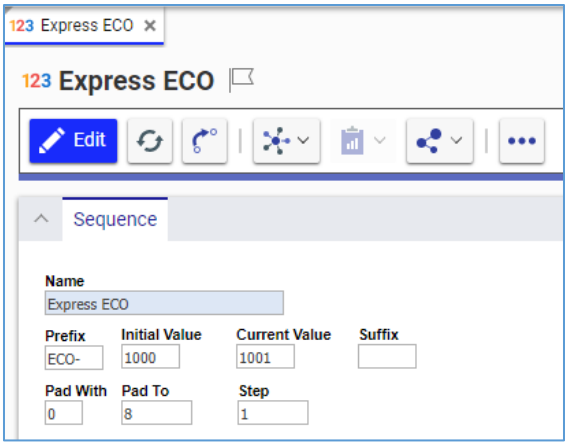

Figure 12.

To define a template, the **Sequence** Item view provides the following fields:

- **Name** – The Sequence Item name.
- **Prefix** – A constant group of alphanumeric characters at the beginning of the generated Item numbers. For example, in the **ECO-**xxxx1001-AC Item number, **ECO-** is the **Prefix**.
- **Initial Value** – The numeric value that begins the Sequence.
- **Current Value** – The last-assigned numeric value in a Sequence. An Item number to be generated next will be **Current Value** incremented by **Step**. For example, if ECO-xxxx**1001**-AC is the last number and Step is 1, then the next sequence will be ECO-xxx**1002**-AC.
- **Suffix** – A fixed group of alphanumeric characters at the end of the generated Item numbers. For example, in the ECO-00001001**-AC** Item number, **-AC** is **Suffix**.
- **Pad With** – A character filling the required number of spaces. For example, the ECO-***xxxx*1001**-AC Item number has eight sequential numbers, four of which are unused and filled with **x.** Thus, **Pad With** is **x**.
- **Pad To** – The total number of characters for the number excluding the Prefix and Suffix. For example, the ECO-**xxxx1001**-AC Item number has eight characters. Thus, **Pad To** is **8**.
- **Step** – A numeric value by which the sequential numbers of an Item number are incremented.

To access or create a Sequence, click the Navigation button  and select **Administration → Sequences**.

To use a Sequence on an ItemType, set the **Data Type** as **Sequence** and the **Data Source** as ***Sequence Item Name*** on the **item_number** property.

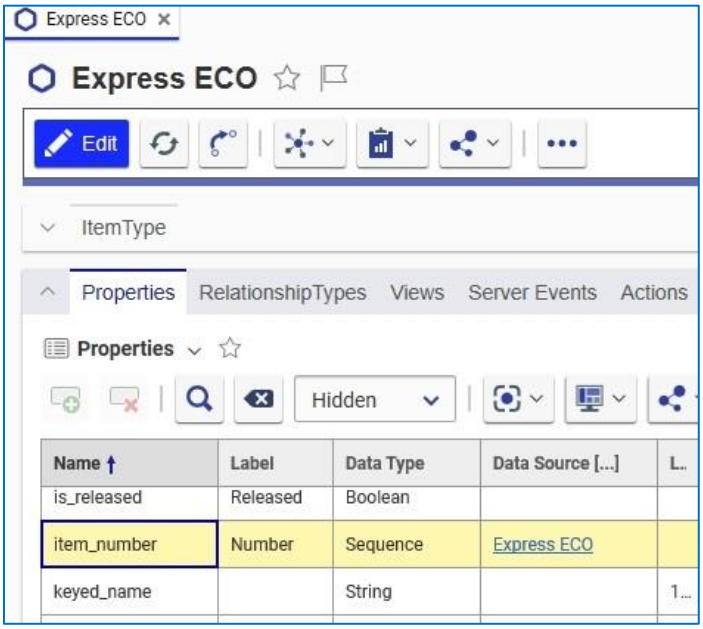

Figure 13.

5 Configuring Change Management

The CMII, Simple Change Management, and Express Change Management processes are available in the Product Engineering application. Each process has its own Change Management ItemTypes. They are discussed in detail in the *Aras Product Engineering 14 - User Guide*.

As an Administrator, you can use the **Choose CM Options** dialog to allow one or more of these change processes (ItemTypes) to run in parallel as well as to set the default Change Management ItemType for the affected Items: Parts, Documents, and CAD Documents.

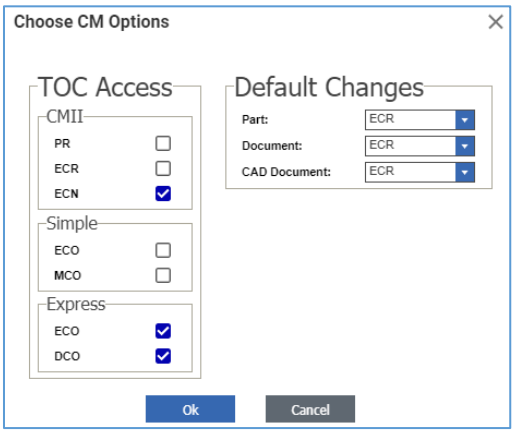

Figure 14.

To access the **Choose CM Options** dialog, click the **User Menu** icon  and select **Actions** → **Choose CM Options**.

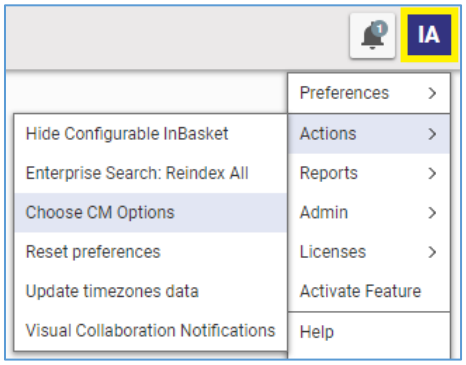

Figure 15.

5.1 Configuring Change Management TOC Access

The **Choose CM Options** dialog provides a convenient tool to specify the visibility of Change Items in the TOC. The **TOC Access** group of check boxes displays Change ItemTypes:

- By selecting ***a check box***, the corresponding ItemType will be visible on the TOC for the **All Employees** Identity.
- By clearing ***a check box***, the corresponding ItemType will be hidden on the TOC for all Identities except for **Aras PLM**.

If a more elaborate TOC Access is needed, you can use the standard TOC configuration.

5.2 Setting up a Default Change Management Item

One of the ways to initiate a change for an affected Item is to access the **Choose Change Item** dialog: right-click the ***Item*** in the *search grid* and then click **Add Item(s) To Change** in the dropdown menu.

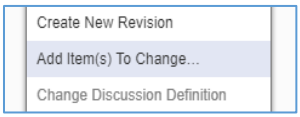

Figure 16.

By default, the **Choose Change Item** dialog is opened with **ECR** for all affected ItemTypes.

Figure 17.

The users use the **Change Type** drop-down list to select another Change Item.

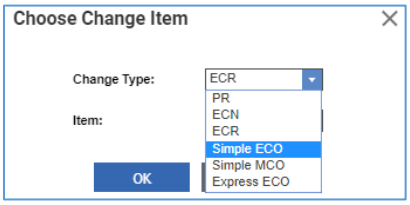

Figure 18.

As an Administrator, you can configure the **Choose Change Item** dialog to be opened with a default Change ItemType for a given affected ItemType. Use drop-down lists in the **Default Changes** group of the **Choose CM Options** dialog.

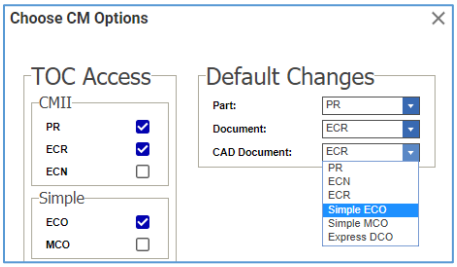

Figure 19.

Configuring the **Default Changes** group does not affect TOC Access.

6 Configuring Effectivity Management

Effectivity Management of Part BOM structures is available in the Product Engineering (PE) application. This enables expressing Effectivity conditions of Parts within the BOM and resolving BOM structures against these conditions. As an Administrator, you can configure and customize some aspects of Effectivity Management in the PE application, such as:

- Creating additional Effectivity Variables and Effectivity Scopes.
- Customizing the Effectivity Scope Builder Method for custom business logic.
- Configuring Query Definitions for structure resolution logic.
- Configuring Tree Grid Views to display Item and Relationship properties.
- Configuring Permissions that determine who can manage Effectivity information.

Note: The **effs_** name prefix is used for the ItemTypes that belong to Effectivity Management.

6.1 Overview of the Effectivity Services Data Model

This section provides a high-level overview of the Effectivity Services based data model concept as implemented in the PE application. For more information on Effectivity Services, refer to the *Aras Innovator – Effectivity Services Programmer's Guide*.

An Effectivity Scope defines a context for Resolution through Effectivity Criteria, which should be applied to particular effective items. An Effectivity Scope consists of the following items:

- Effectivity Variables, which are variables influencing Effectivity decisions.
- Effectivity Scope ItemType, which determines Items to be resolved by Effectivity.

A Builder Method uses Effectivity Variables to construct a Scope object in the runtime, used by the Effectivity Resolution Engine to perform Resolution of effective items.

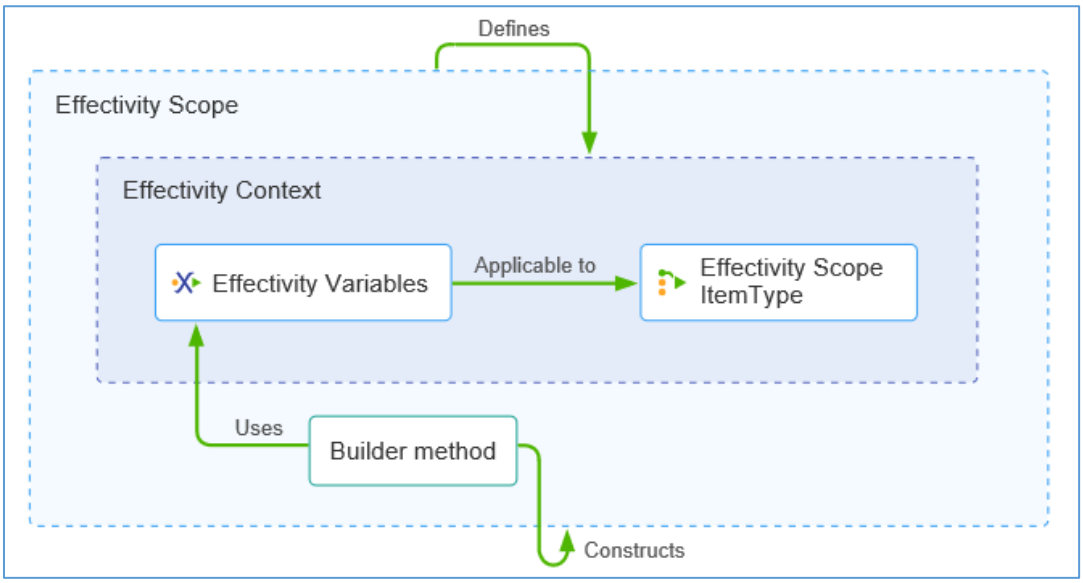

Figure 20.

6.2 Overview of Effectivity Services, TGV, and QD Interactions

The user interacts with the Tree Grid View (TGV) to provide the Effectivity Criteria for structure resolution.

TGV requests necessary data from the associated Query Definition (QD). The Effectivity Criteria is passed to this QD via QD parameters.

QD requests Effectivity Expression items with provided Effectivity Criteria.

The Effectivity Resolution Engine resolves Effectivity Expression items against provided Effectivity Criteria and returns the result.

The results are displayed to the user using the TGV.

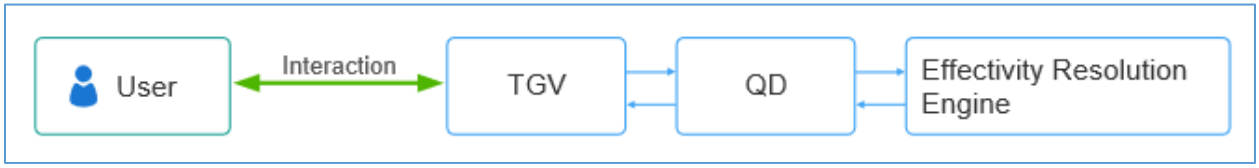

Figure 21.

6.3 Managing Effectivity Variables

Many variables influence Effectivity decisions. They depend on the industry, product, and business processes among others. They are used in:

- Effectivity Expressions to set Effectivity conditions within a structure.
- Effectivity Criteria for structure resolution.

Some of the most commonly used Effectivity Variables are provided out of the box:

- Model
- Unit
- Date

The **Aras Part BOM Scope** Effectivity Scope uses these Effectivity Variables.

Based on business needs, additional Effectivity Variables can be defined, as described in the following section.

To access the Effectivity Variables, select **Administration → Effectivity Services → Effectivity Variables** from the TOC.

6.3.1 Creating Effectivity Variables

Use the standard Item creation procedure to create a new Effectivity Variable. The following fields are required:

- **Name**
- **Type**: The out-of-the-box Builder Method supports the following Effectivity Variable types:
  - **Date**
  - **Integer**
  - **Item**

- **List**
- **String**

- If the specified ***Type*** is **List** or **Item**, provide an accompanying value in the **List** or **ItemType** field.

Note: When creating an Effectivity Variable with type “Item”, the ItemType must be added as a Poly Source of the ScopeCachedependency ItemType.

A List used for an Effectivity Variable must have both a Value and a Label for each entry. The Builder Method uses both Value and Label. The Label is presented to a user when working with effectivity.

6.3.2 Editing Effectivity Variables

Warning Before changing any of the required fields of an Effectivity Variable, make sure that the Effectivity Variable is not used in any Effectivity Expressions.

Use the standard Item editing procedure to edit an Effectivity Variable.

6.3.3 Deleting Effectivity Variables

Warning Before deleting an Effectivity Variable, make sure that the Effectivity Variable is not used in any Effectivity Expressions.

Use the following procedure to delete an Effectivity Variable:

1. Remove it from its associated Effectivity Scopes.
2. Use the standard Item deletion procedure to delete the Effectivity Variable.

6.4 Managing Model Items for the Model Effectivity Variable

The **Model** Effectivity Variable uses Items of the **Model** (**effs_model**) ItemType. There are no Model Items out of the box. If your business uses the Model as an Effectivity Variable, you need to create Model Items.

To access the Model Items, go to **Administration** → **Effectivity Services** → **Models** in the TOC.

6.4.1 Creating a Model Item

Use the standard Item creation procedure to create a new Model Item. The **Name** field is required and unique.

Note: If a Model has a Label, then the Label is presented to a user when working with effectivity. Otherwise, the Name is presented.

6.4.2 Editing a Model Item

Warning Before changing the Name or Label of a Model Item, make sure that the Model Item is not used in any Effectivity Expressions**.**

Use the standard Item editing procedure to edit a Model Item.

6.4.3 Deleting a Model Item

Warning Before deleting a Model Item, make sure that it is not used in any Effectivity Expressions.

Use the standard Item deleting procedure to delete a Model Item.

6.5 Managing Effectivity Scopes

An **Effectivity Scope** is an Item of the **effs_scope** ItemType that defines the context for setting Effectivity conditions and resolving structures against given Effectivity Criteria.

To access the Effectivity Scopes, select **Administration → Effectivity Services → Effectivity Scope** from the TOC.

The PE application provides the **Aras Part BOM Scope** Effectivity Scope. The **effs_scopeObjectBuilderMethod** Builder Method constructs an object of this Scope.

The **Aras Part BOM Scope** Effectivity Scope uses the **Part BOM** ItemType and the **Model**, **Unit**, and **Date** Effectivity Variables. You can either update this Scope or create a new one to specify a set of Effectivity Variables that are relevant to your business.

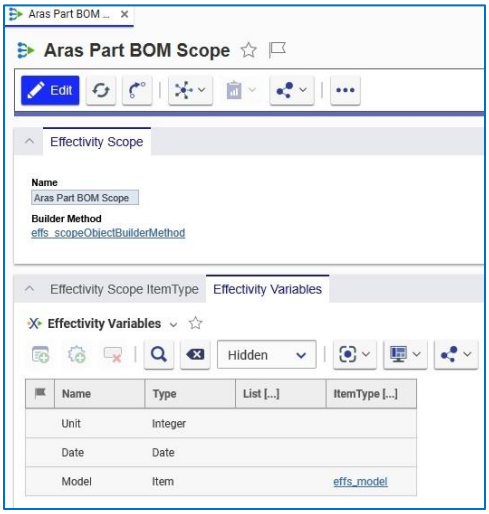

Figure 22.

6.5.1 Builder Method

A Builder Method is an Item of the **Method** ItemType, which constructs the Scope object using business data and logic.

In the PE application, **effs_scopeObjectBuilderMethod** is the default Builder Method that works with the Effectivity Scope ItemType and Effectivity Variables.

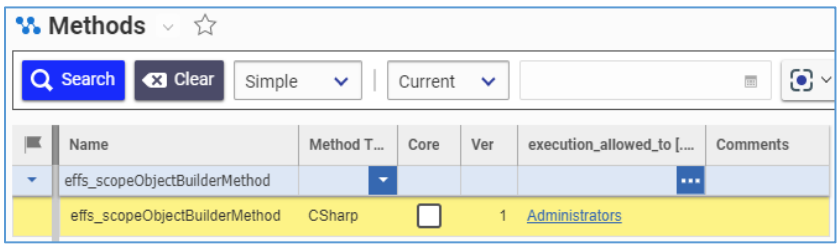

Figure 23.

6.5.2 Managing Effectivity Variables in Effectivity Scopes

There are no restrictions on the number or types of Effectivity Variables in an Effectivity Scope. You can either update the **Aras Part BOM Scope** Effectivity Scope or create a new Effectivity Scope to specify a set of Effectivity Variables that are relevant to your business.

6.5.2.1 Attaching Effectivity Variables to Effectivity Scopes

Use the standard procedure for attaching Items on the **Effectivity Variables** accordion tab of ***an existing Effectivity Scope*** Item view.

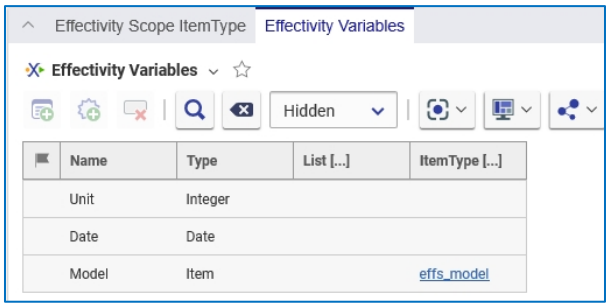

Figure 24.

6.5.2.2 Removing Effectivity Variables from Effectivity Scopes

Warning Do not remove an Effectivity Variable from an Effectivity Scope if it is used in an Effectivity Expression for the given Scope.

Use the standard procedure for removing attached Items on the **Effectivity Variables** accordion tab of ***an Effectivity Scope*** Item view.

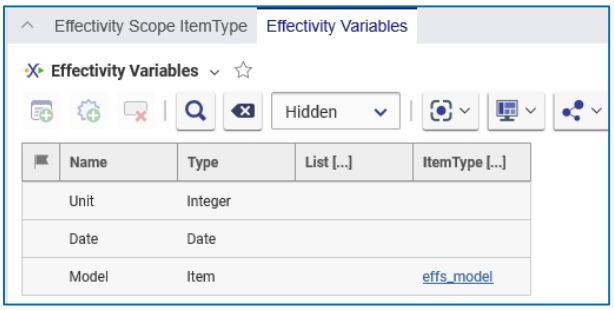

Figure 25.

6.5.3 Creating Effectivity Scopes

Use the standard Item creation procedure to create a new Effectivity Scope. The following fields are required:

- **Name**
- **Builder Method** -- Use the **effs_scopeObjectBuilderMethod method** unless you have implemented a different Method that uses custom business logic.

Specify Effectivity Scope ItemTypes and Effectivity Variables on the corresponding accordion tabs.

6.5.4 Finding Where Effectivity Scope is Used

To find out where an Effectivity Scope is used, you can use the **Where Used** browser.

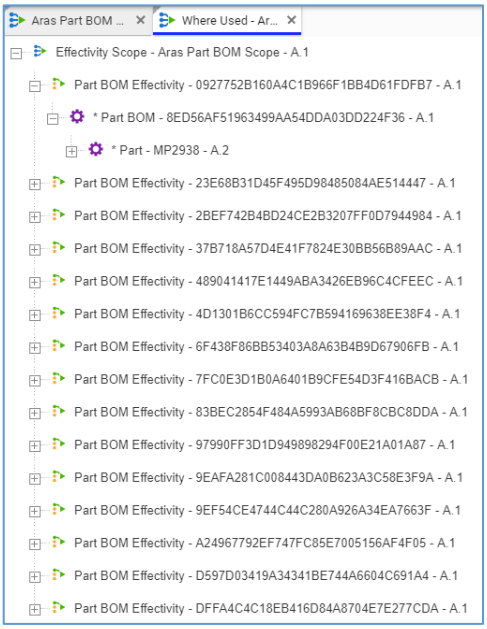

Figure 26.

6.6 Understanding Query Definitions for Effectivity Resolution

This section provides an overview of the Query Definitions (QD) configuration for resolving the BOM structures against Effectivity Criteria.

- Definition of a QD containing Effectivity filtering logic.
- Merging this QD into another QD to create a single executable QD for a related Tree Grid View.

In the Product Engineering (PE) application, Part BOM Relationships are resolved against Effectivity Criteria on the **BOM Structure** tab of a Part Item. The resolution uses Effectivity Scope and Effectivity Variable values set in the **Effectivity Criteria Filter** dialog. The resolved structure is displayed using the **PE_BomStructure** Tree Grid View (TGV) configuration.

You can configure the **PE_BomStructure** TGV to add, remove, or reorder columns to display Part and Part BOM properties. Adding or removing columns requires changing the QD configuration to return the given properties. You can use the standard TGV and QD procedures for this configuration.

The QD containing Effectivity filtering logic is **effs_effectivityResolutionOnPartBom**. It retrieves assigned Effectivity Expressions and evaluates them against the given Effectivity Criteria in the Resolution Engine. The QD filters out Items that are evaluated as not valid for the given Effectivity Criteria.

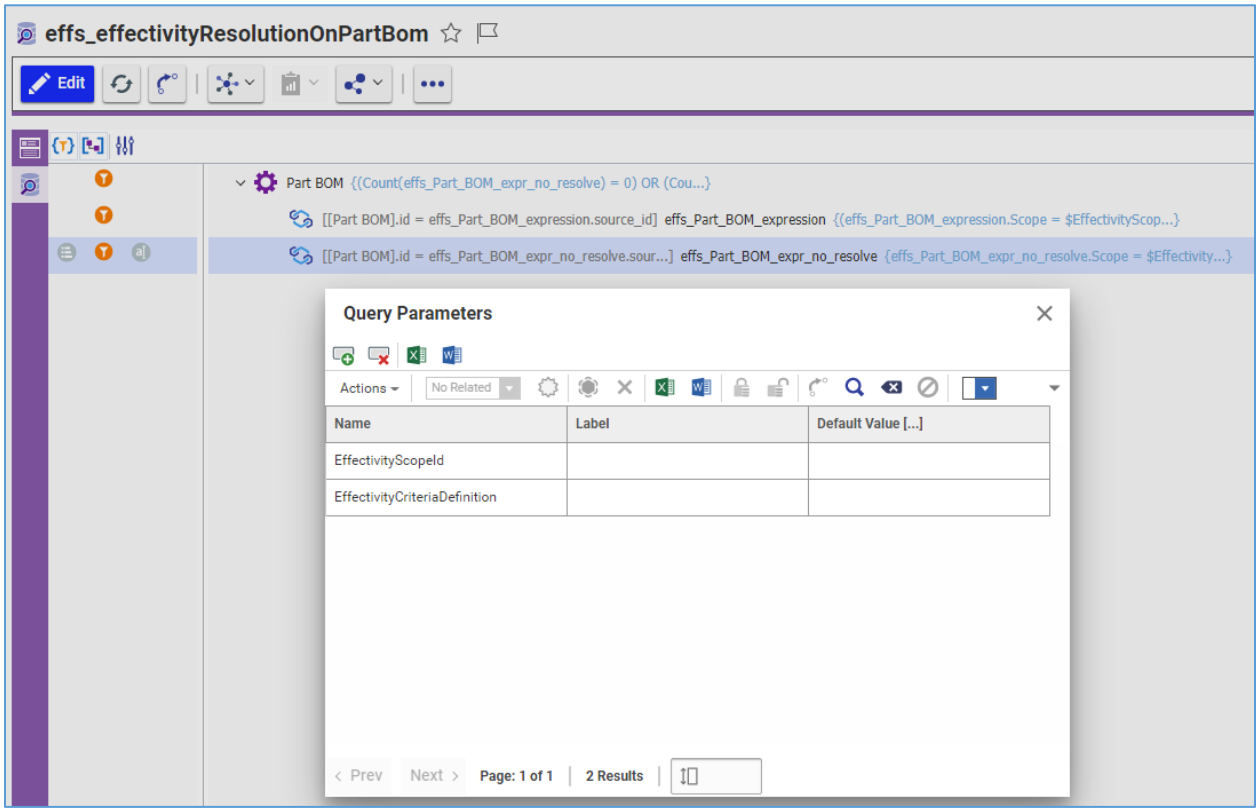

Figure 27.

The **effs_effectivityResolutionOnPartBom** QD passes data to the Effectivity Resolution Engine using the following Query Parameters:

- **EffectivityScopeId**—the ID of the Effectivity Scope.
- **EffectivityCriteriaDefinition**—the Effectivity Criteria Definition.

The Query Structure of the **effs_effectivityResolutionOnPartBom** QD is as follows:

- **Part BOM** is the Effective ItemType of the PE application Effectivity Scopes.

  Where Condition:

```
(Count(effs_Part_BOM_expr_no_resolve) = 0) OR
(Count(effs_Part_BOM_expression) > 0)
```

Note: You can change the **Where Condition** to use AND logic instead of OR. The logic to use depends on how multiple Effectivity Expressions on a given single Part BOM Item should be evaluated. For details, refer to section *7.9 Configuring Logic to Evaluate Effectivity Expressions for Effectivity Resolution*.

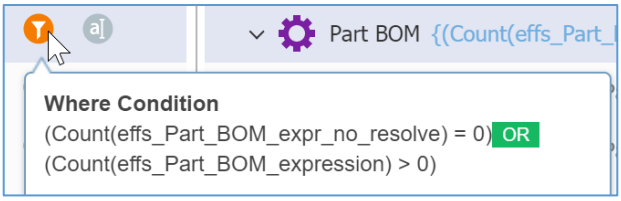

Figure 28.

Note: It is recommended to use only the “Count()” aggregate function for Effectivity relationships to prevent performance degradation.

- **effs_Part_BOM_expression** is a RelationshipType with the source Part BOM ItemType.

  Where Condition:

  ```
  (Scope = $EffectivityScopeId) AND (Definition =
  '<expression>$EffectivityCriteriaDefinition</expression>')
  ```

  The Effectivity Scope ID value is set from the **Scope** cell, and the Effectivity Criteria Definition value is set from the **Variables** and **Values** cells of the **Effectivity Criteria Filter** dialog.

  Join Condition:

  ```
  [Part BOM].id = effs_Part_BOM_expression.source_id
  ```

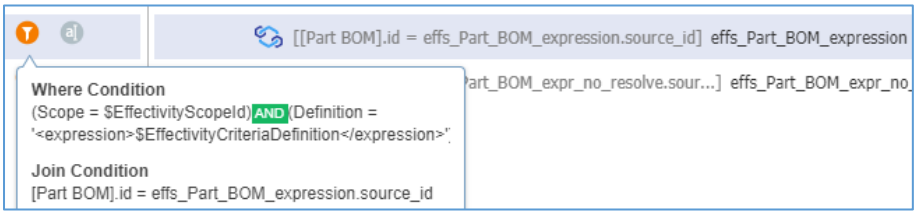

Figure 29.

- **effs_Part_BOM_expr_no_resolve** is used to check if the Part BOM item has any effectivity relationship items for the specified scope.

  Where Condition:

  ```
  Scope = $EffectivityScopeId
  ```

  The Effectivity Scope ID value is set from the **Scope** cell.

  Join Condition:

  ```
  [Part BOM].id = effs_Part_BOM_expr_no_resolve.source_id
  ```

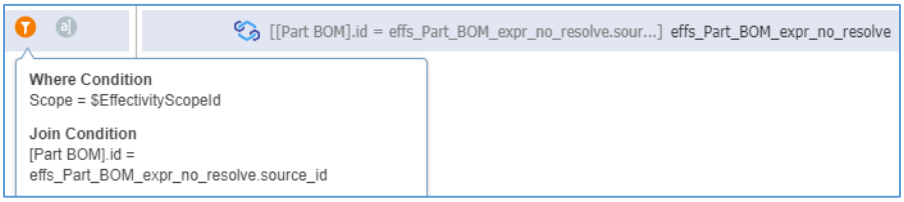

Figure 30.

The **effs_effectivityResolutionOnPartBom** QD is merged into the **PE_BomStructure** QD to produce a single executable QD, as described later in this section.

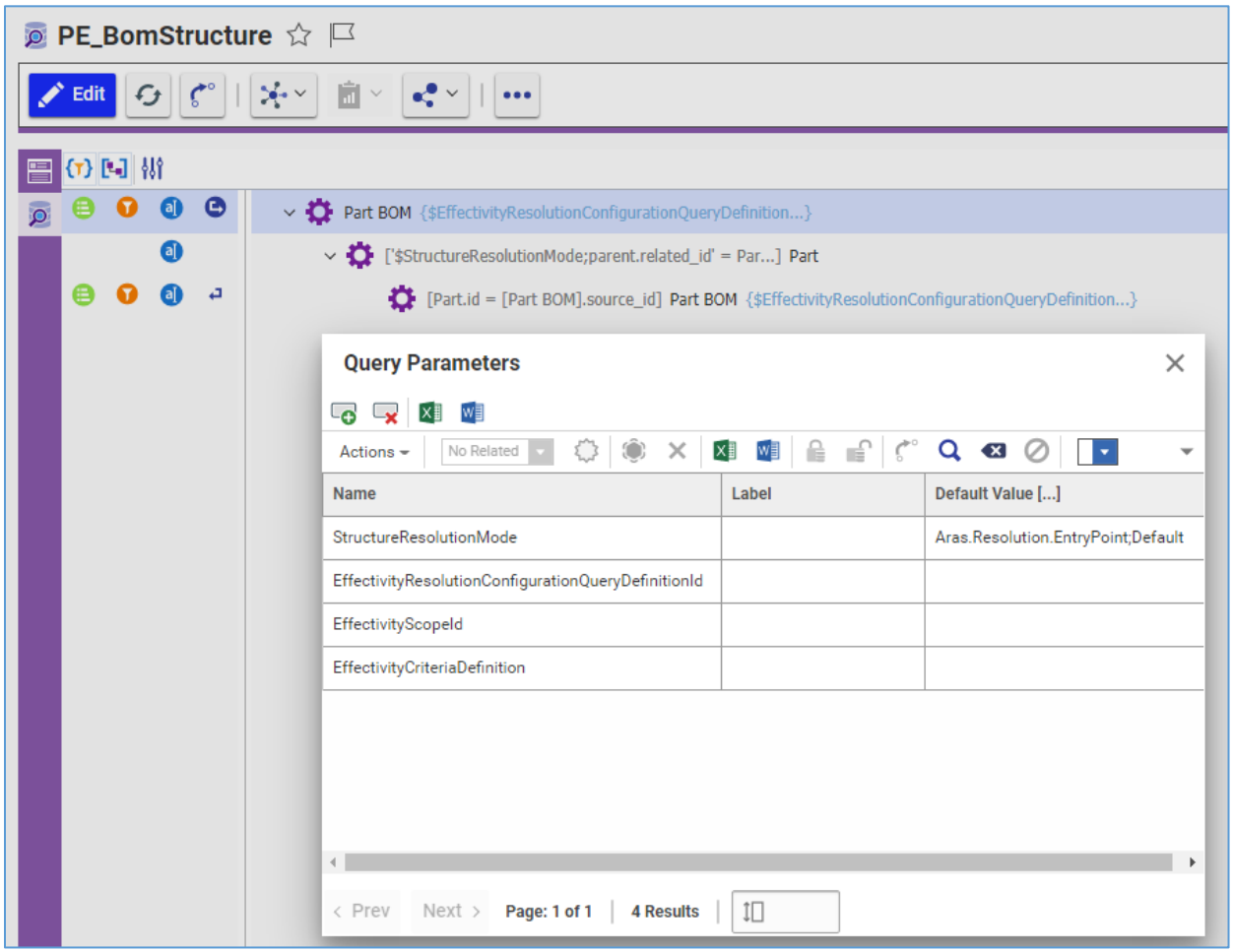

Figure 31.

The **PE_BomStructure** QD has the following Query Parameters:

- **EffectivityResolutionMode** — structure resolution mode. **Aras.Resolution.EntryPoint;Default** is the default value.
- **EffectivityResolutionConfigurationQueryDefinitionId** — ID of the QD containing Effectivity filtering logic to enable Effectivity resolution.
- **EffectivityScopeId** — ID of the Effectivity Scope Item passed to the Effectivity Resolution Engine.
- **EffectivityCriteriaDefinition** — the Effectivity Criteria Definition passed to the Effectivity Resolution Engine.

When Effectivity Resolution is triggered from the **Effectivity Criteria Filter** dialog (see section *6.7 Understanding TGV Definition for Effectivity Resolution*), the **EffectivityResolutionConfigurationQueryDefinitionId** parameter value is retrieved from the **effs_getResolutionCfgQdIdByTgvId** Method.

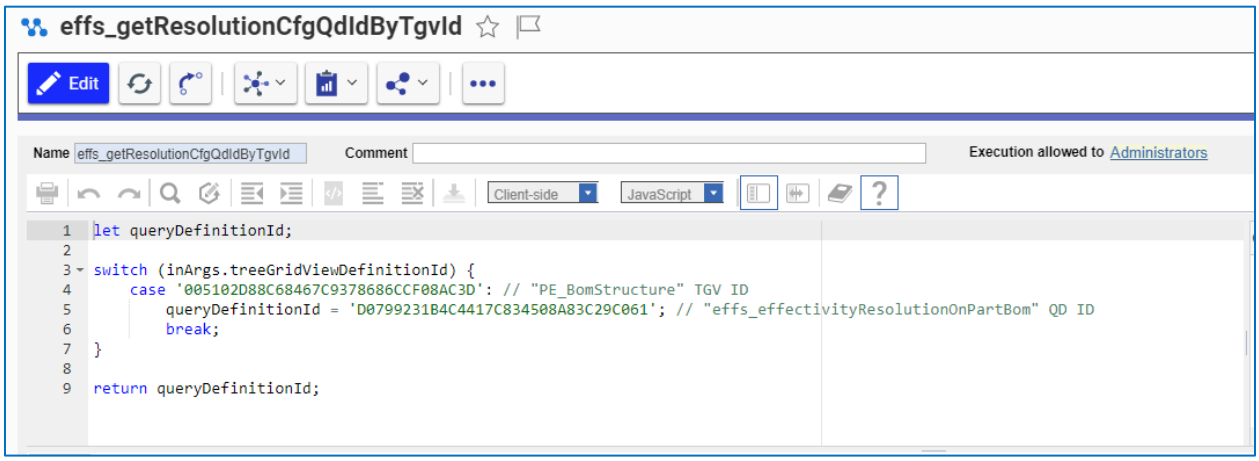

Figure 32.

The method defines 1-to-1 associations between Tree Grid Views and Query Definitions containing Effectivity Resolution logic to be merged for Effectivity Resolution, when triggered from the **Effectivity Criteria Filter** dialog.

The Query structure of the **PE_BomStructure** QD is recursive:

- **Part BOM** is the root ItemType of the **BOM Structure** accordion tab.

  Order:

  ```
  1. Sequence (Ascending)
  ```

  Where Condition:

  ```
  $EffectivityResolutionConfigurationQueryDefinitionId =
  $EffectivityResolutionConfigurationQueryDefinitionId
  ```

Note: This condition is used as an anchor to detect the place in the QD where the **effectivityResolutionOnPartBom** QD is merged. This condition should not be removed otherwise Effectivity Resolution will not work.

Properties:

```
Quantity, Sequence, Reference Designator, Expression, id
```

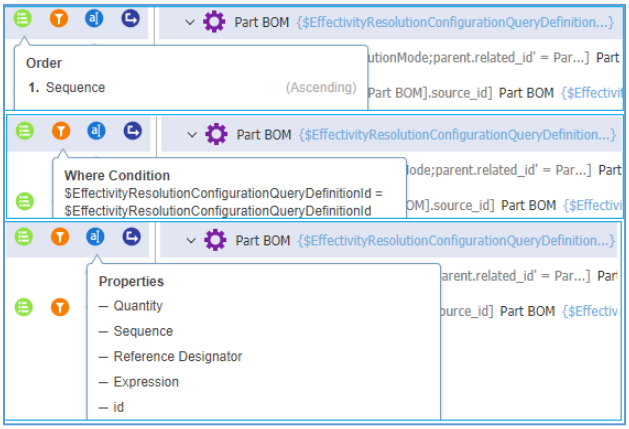

Figure 33.

- **Part**

  Join Condition:

  ```
  '$StructureResolutionMode;parent.related_id' = Part.id
  ```

  Properties:

  ```
  Part Number, Revision, State, locked_by_id, Name, id
  ```

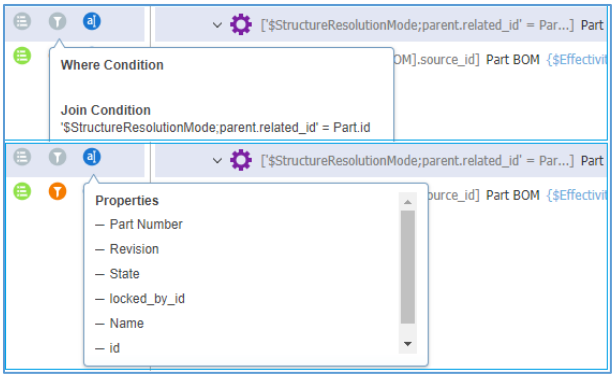

Figure 34.

- **Part BOM** is extracted recursively – all its values are the same as for the top **Part BOM**, and the following **Join Condition** is added:

```
Part.id = [Part BOM].source_id
```

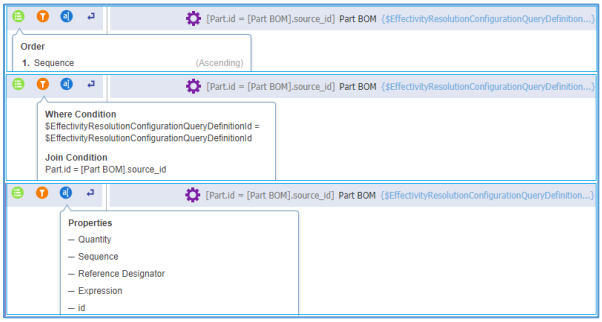

Figure 35.

To merge the QD for Effectivity Resolution, the **PE_BomStructure** QD has the **OnBeforeExecute** event set for the **effs_mergeQueryDefinition** Method.

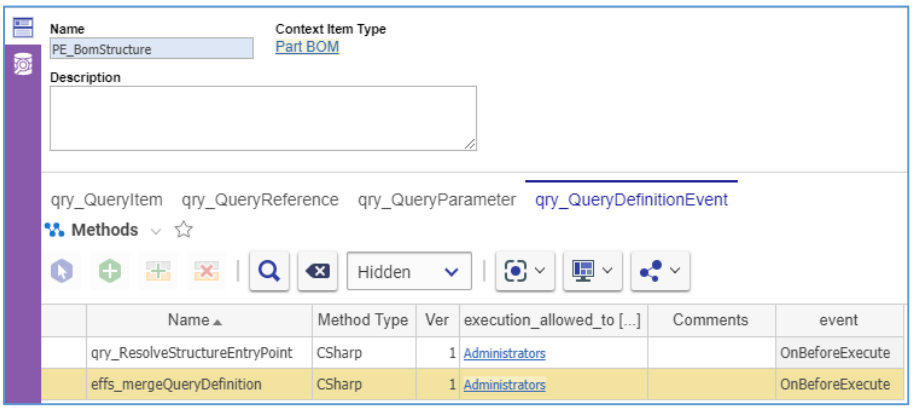

Figure 36.

By default, the QD Item view does not display tabs. To view and to add events to QDs, make tabs visible by setting the **Default Structure View** property of the **qry_QueryDefinition** ItemType to **Tabs On**.

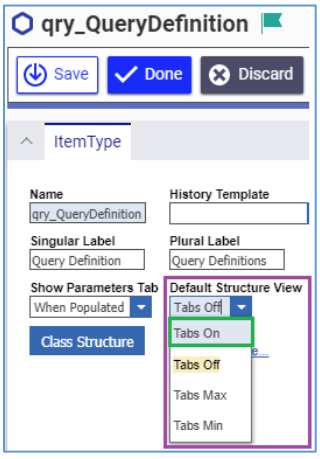

Figure 37.

6.7 Understanding TGV Definition for Effectivity Resolution

This section provides an overview of how Tree Grid View (TGV) is configured to work with a corresponding Query Definition (QD) to display the resolved structure for user-provided Effectivity Criteria.

In the Product Engineering (PE) application, the Part BOM Relationships are resolved against Effectivity Criteria on the **BOM Structure** tab of a Part Item. This tab uses a TGV to display data retrieved from a QD as a tree grid.

You can configure the **PE_BomStructure** TGV to add, remove, or reorder columns to display Part and Part BOM properties. Adding or removing columns requires changing the QD configuration to return the given properties. You can use the standard TGV and QD procedures for this configuration.

The **PE_BomStructure** TGV configuration has the following UI elements: the **Set Effectivity Criteria** button, the **View Effectivity** context menu item, and the **Manage Effectivity Expressions** split pane.

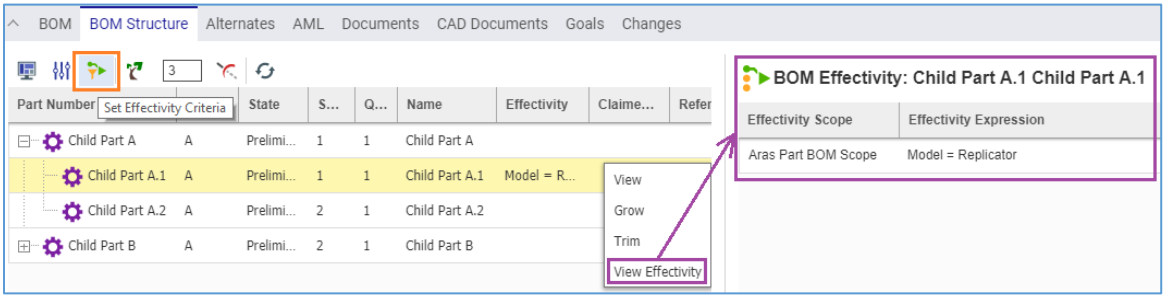

Figure 38.

You can configure the **PE_BomStructure** TGV to add, remove, or reorder columns to display Part and Part BOM properties. Adding or removing columns requires changing the QD configuration to return the given properties. Use the standard TGV and QD configuration procedures.

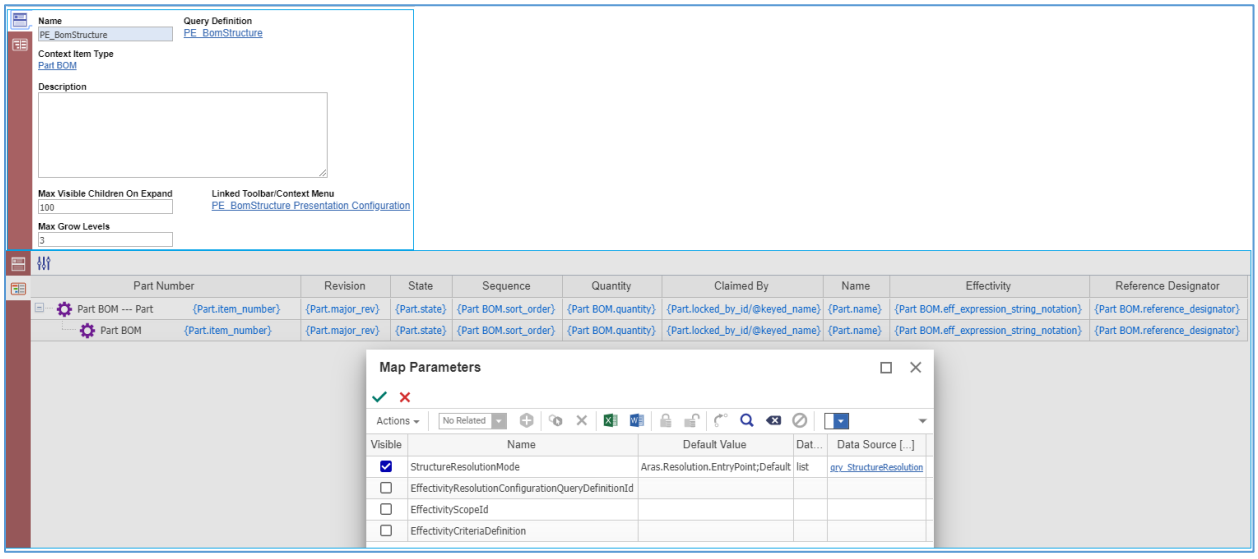

Figure 39.

The TGV configuration uses the **PE_BomStructure Presentation Configuration**, which adjusts the toolbar and context menu with two new command bar sections: **effs_tgvToolbarCustomization** and **effs_tgvContextMenuCustomization**.

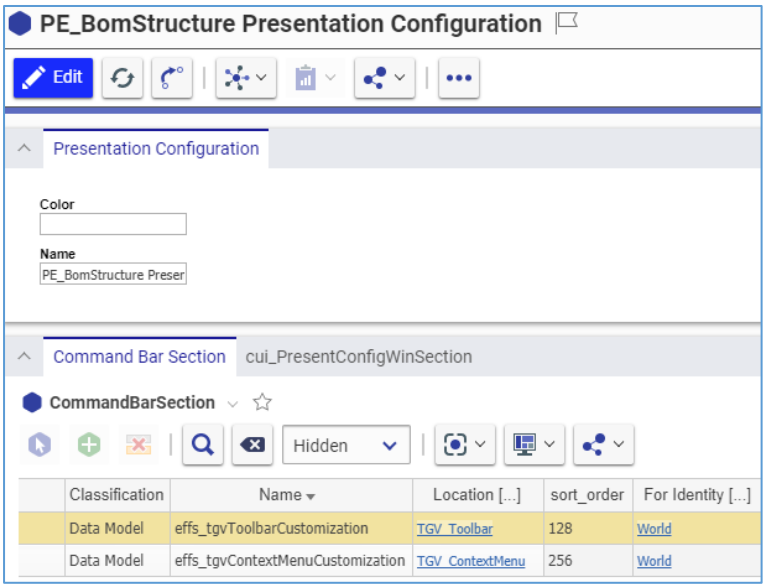

Figure 40.

The **effs_tgvToolbarCustomization** command bar section adds the new button to the default TGV toolbar. This button is **effs_tgvToolbarSetEffectivityCriteriaButton** labeled as **Set Effectivity Criteria**.

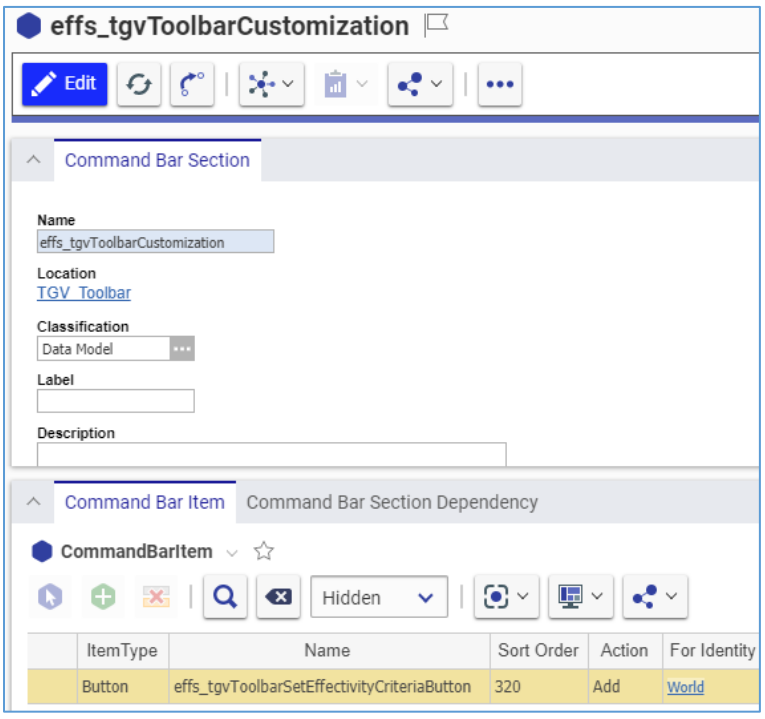

Figure 41.

Set Effectivity Criteria triggers the **effs_setEffectivityCriteriaHandl** Method to show the **Effectivity Criteria Filter** dialog for setting the Effectivity Criteria, for which a BOM structure will be resolved.

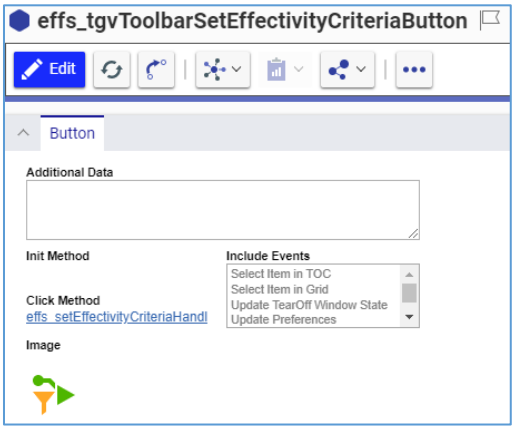

Figure 42.

The **effs_tgvContextMenuCustomization** command bar section adds a new menu item to the default TGV context menu. This menu item is **effs_tgvContextMenuViewEffectivityItem** labeled as **View Effectivity**.

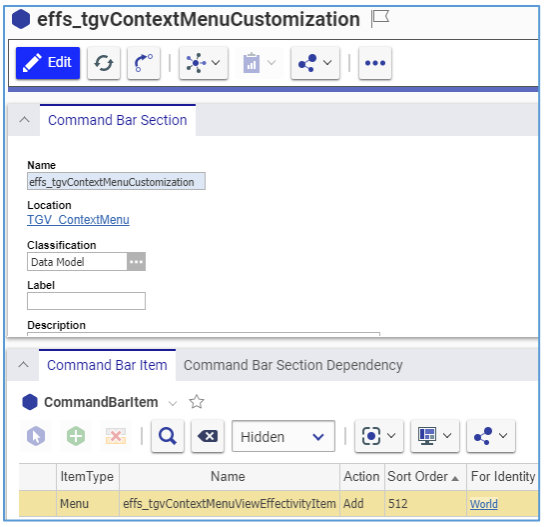

Figure 43.

View Effectivity triggers the **effs_bomStructureViewEffCtxHandl** Method to display the **Manage Effectivity Expressions** split pane with Effectivity Expressions set for the given Part BOM.

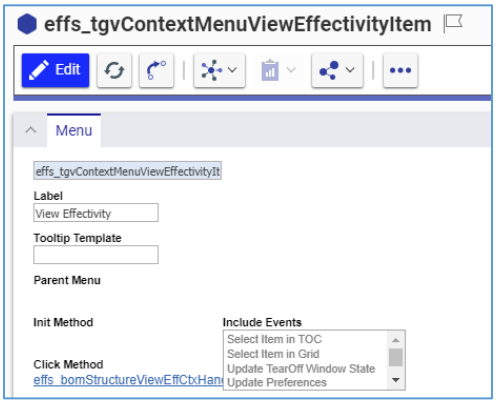

Figure 44.

6.8 Controlling the Visibility of Effectivity

In Aras Innovator, Effectivity Management on Part BOM structures is available out of the box. If your business is not going to use this feature, you can hide its associated UI elements. This section explains how to hide and redisplay the discussed UI elements. These elements are as follows:

- The **Effectivity Services** subcategory under **Administration** in **Contents**:
  - **Effectivity Scope**
  - **Effectivity Variables**
  - **Models**
- The **Effectivity** tab in the **Part BOM** Relationship Item view.
- The **BOM Structure** tab in the **Part** Item view:
  - The **Set Effectivity Criteria** button in the **BOM Structure** toolbar
  - The **View Effectivity** menu item in the context menu of a Part
  - The **Effectivity** column

Note: Hiding Effectivity-related UI elements does not delete any existing Effectivity Items. If you redisplay the UI elements, the Effectivity Items will be available.

6.8.1 Visibility of the Effectivity Services TOC Subcategory

The **Effectivity Services** subcategory of the **Administration** category in the **Table of Contents** (TOC) contains the following ItemTypes:

- **effs_scope** (**Effectivity Scope**)
- **effs_variable** (**Effectivity Variable**)
- **effs_model** (**Model**)

The visibility of the **Effectivity Services** category depends on the visibility of its contained ItemTypes:

- Visible when at least one of the ItemTypes is visible.

- Hidden when all the ItemTypes are hidden.

You can hide and redisplay the **Effectivity Services** ItemTypes in any sequence. Use the standard TOC configuration.

When redisplaying an **Effectivity Services** ItemType, use the **Effectivity Management** Identity and **Effectivity Services** Category.

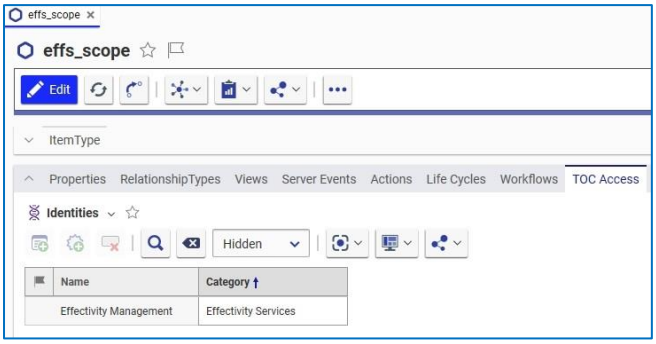

Figure 45.

6.8.2 Visibility of the Effectivity Tab in Part BOM Item View

By default, the **Part BOM** Relationship Item view includes the **Effectivity** accordion tab.

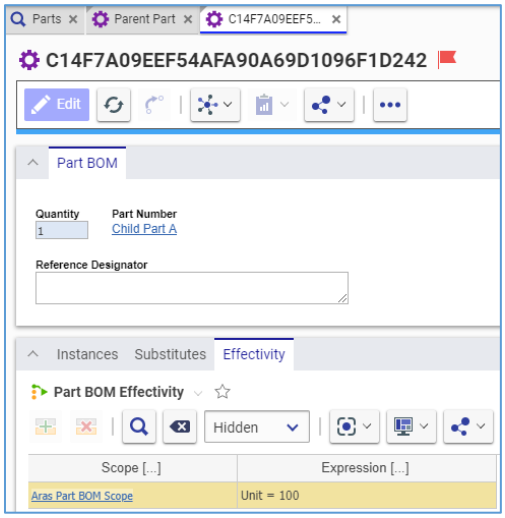

Figure 46.

The visibility of the **Effectivity** accordion tab is controlled by the **Hide In All** check box in the **Source** group of the **effs_Part_BOM_expression** RelationshipType Item view.

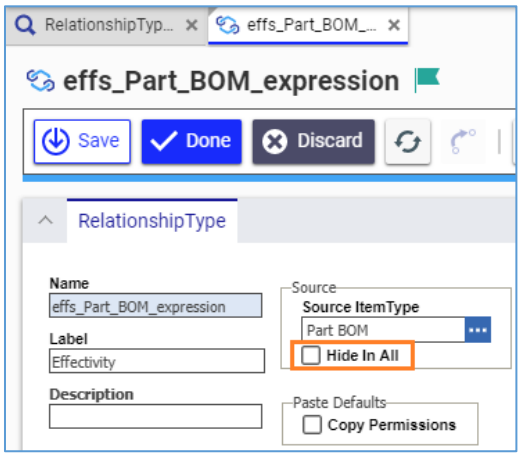

Figure 47.

To hide this tab, select the **Hide In All** check box. To display this tab, clear the check box.

6.8.3 Visibility of the Effectivity-Related Elements in Part Item View

By default, the **BOM Structure** accordion tab of the **Part** Item view has the following Effectivity Management UI elements:

- The **Set Effectivity Criteria** button in the **BOM Structure** toolbar.
- The **View Effectivity** menu item in the context menu of a Part.
- The **Effectivity** column.

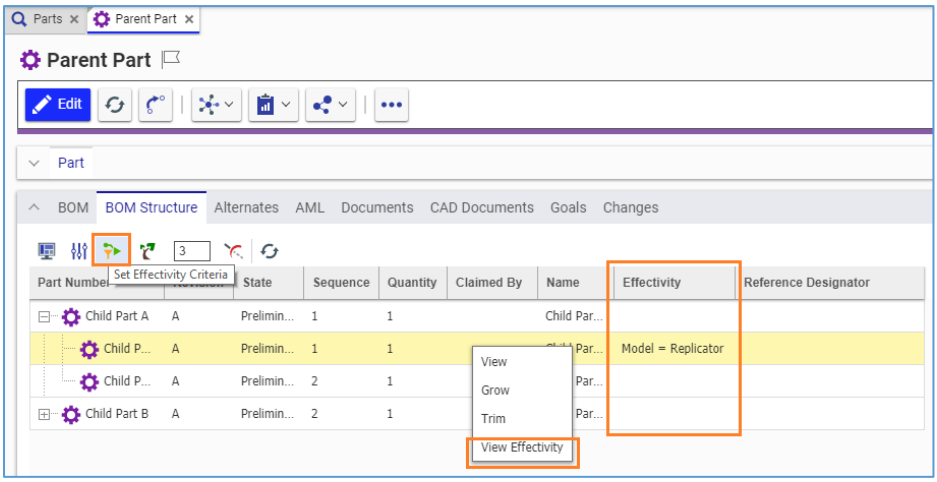

Figure 48.

This section describes configuration specifications for the **PE_BomStructure** QD and TGV, which shows these UI elements as well as calls Effectivity filtering.

If you need to hide the discussed elements, remove the described configuration from the TGV and then from the QD. If you need to redisplay the discussed elements after you have removed them, configure the QD and then the TGV according to the given specifications. Relog into Aras Innovator to confirm the change.

6.8.3.1 PE_BomStructure TGV Configuration for Visibility of Effectivity-Related Elements

To show the Effectivity-related UI elements on the **BOM Structure** accordion tab of the **Part** Item view, the **PE_BomStructure** TGV is configured as specified in this section. The **PE_BomStructure** TGV uses the **PE_BomStructure Presentation Configuration**.

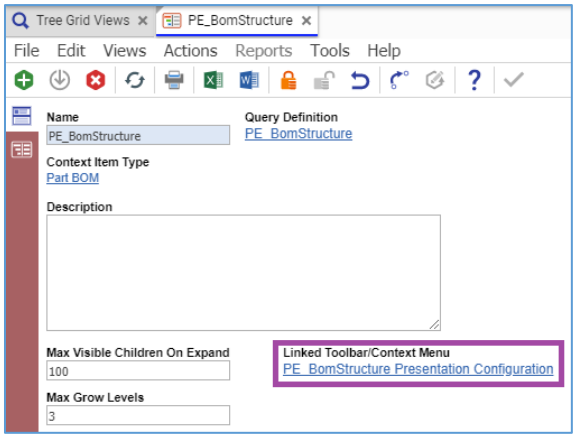

Figure 49.

The **PE_BomStructure Presentation Configuration** is configured to show the **Set Effectivity Criteria** toolbar button and the **View Effectivity** context menu item to the **World** Identity:

- **effs_tgvToolbarCustomization** located on **TGV_Toolbar** with sort order **128**.
- **effs_tgvContextMenuCustomization** located on **TGV_ContextMenu** with sort order **256**.

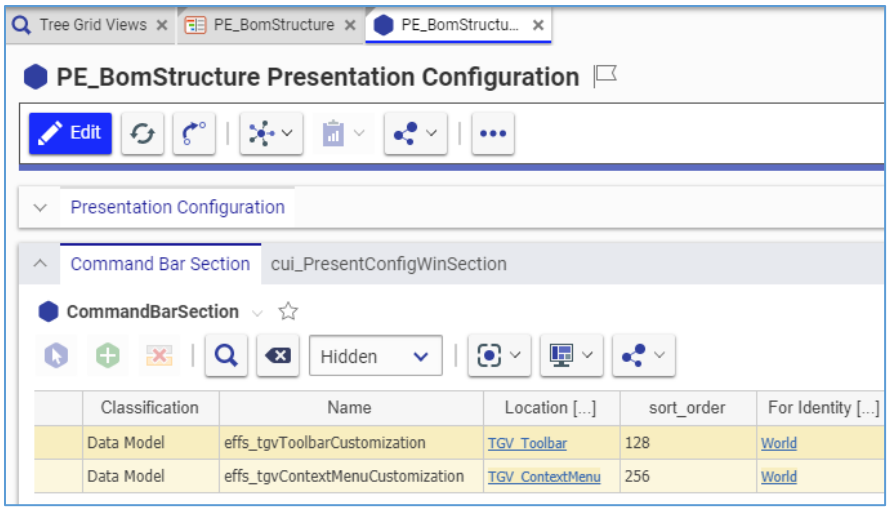

Figure 50.

To display the **Effectivity** column on the **BOM Structure** accordion tab of the **Part** Item View, the **PE_BomStructure** TGV has the **Effectivity** column between the **Name** and **Reference Designator** columns specified to show data of the **Text** type according to the **{Part BOM.eff_expression_string_notation}** text template.

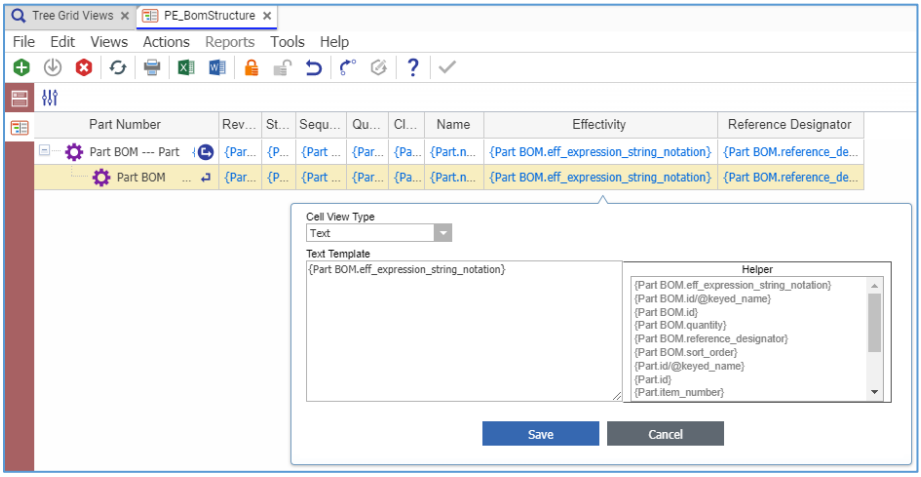

Figure 51.

To hide the Effectivity-related UI elements on the **BOM Structure** tab of the **Part** Item View, remove **effs_tgvToolbarCustomization** and **effs_tgvContextMenuCustomization** from the **PE_BomStructure Presentation Configuration** as well as the **Effectivity** column in the TGV editor, and then reconfigure the **PE_BomStructure** QD. To redisplay these elements after they were hidden, reconfigure the **PE_BomStructure** QD, and then add the command bar sections and Effectivity column as specified.

6.8.3.2 PE_BomStructure QD Configuration for Visibility of Effectivity-Related Elements

To show the Effectivity-related UI elements on the **BOM Structure** accordion tab of the **Part** Item view, the **PE_BomStructure** QD is configured as follows:

- The **qry_QueryDefinitionEvent** relationship tab has an **OnBeforeExecute** event using the **effs_mergeQueryDefinition** Method.

Note: For more information about this Method and how to display and hide tabs in the **QD** Item view, refer to section *7.6 Understanding Query Definitions for Effectivity Resolution*.

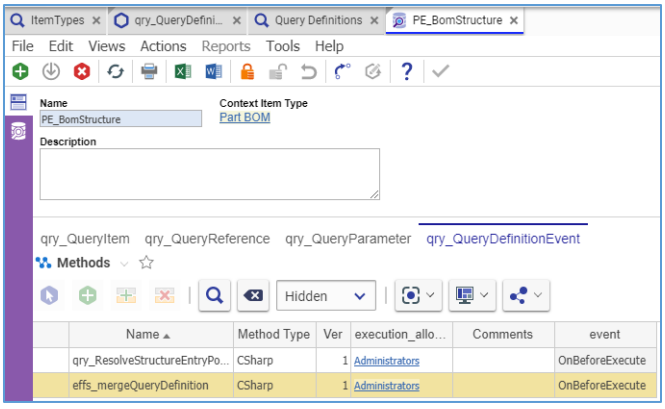

Figure 52.

- The top **Part BOM** item in the QD editor has the **Expression** and **id** properties.

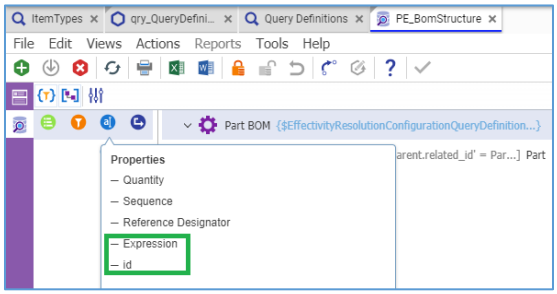

Figure 53.

When hiding the Effectivity-related UI elements on the **BOM Structure** tab of the **Part** Item View, remove the **effs_mergeQueryDefinition** Method and the **Expression** and **id** properties after reconfiguring the **PE_BomStructure** TGV. When redisplaying these elements after they were hidden, add these Method and properties back before reconfiguring the **PE_BomStructure** TGV.

6.9 Configuring Logic to Evaluate Effectivity Expressions for Effectivity Resolution

A Part BOM Item can have several Effectivity Expressions.

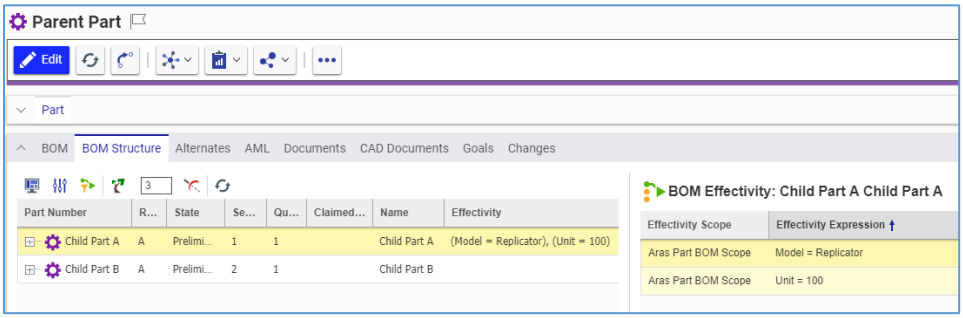

Figure 54.

By default, multiple Effectivity Expressions on a Part BOM Relationship Item are evaluated using the OR conditional operator for the Effectivity Resolution: A given Part BOM is effective against Effectivity Criteria if *at least one* of its Effectivity Expressions evaluates to true for the selected Effectivity Scope.

It is possible to change the evaluation logic to use the AND conditional operator: A given Part BOM is effective against Effectivity Criteria if *all* of its Effectivity Expressions evaluate to true for the selected Effectivity Scope.

The evaluation logic is configured in the **Where Condition** of the top **Part BOM** item in the **effs_effectivityResolutionOnPartBom** QD.

This section describes the configuration specifications of this Query Definition for both evaluation logic.

Note: It is recommended to use only the “Count()” aggreagate function for Effectivity relationships to prevent performance degradation.

6.9.1 Specification of the AND Evaluation Logic

For the AND evaluation logic, the **Where Condition** of the top **Part BOM** item of the **effs_effectivityResolutionOnPartBom** Query Definition is configured as follows:

```
Count(effs_Part_BOM_expression) = Count(effs_Part_BOM_expr_no_resolve)
```

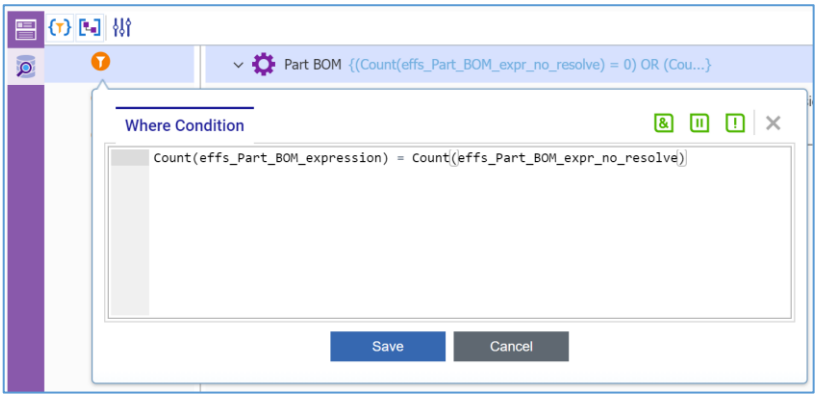

Figure 55.

6.9.2 Specification of the OR (Default) Evaluation Logic

For the OR (default) evaluation logic, the **Where Condition** of the top **Part BOM** item of the **effs_effectivityResolutionOnPartBom** Query Definition is configured as follows:

```
(Count(effs_Part_BOM_expr_no_resolve) = 0) OR
(Count(effs_Part_BOM_expression) > 0)
```

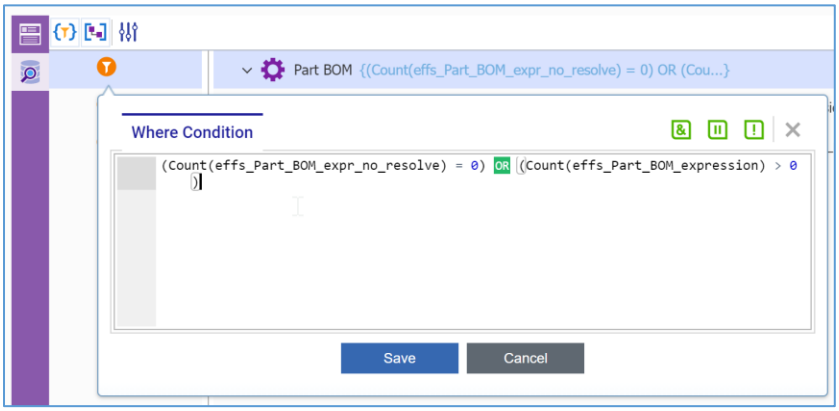

Figure 56.

6.10 Customizing Effectivity String Notations

While an Effectivity Expression is saved as XML, a user-friendly Notation of this Expression is displayed to users. It is possible to change the construction of this Notation. This section describes how the Notation can be customized.

Effectivity on a **Part BOM** Relationship is managed using its related **effs_Part_BOM_expression** Items. A **Part BOM** Relationship can have zero, one, or more Effectivities. The **string_notation** text property on the **effs_Part_BOM_expression** ItemType stores the user-friendly Notation for a single Effectivity Expression. The **eff_expression_string_notation** federated property on the **Part BOM** ItemType dynamically constructs the Effectivity text from its related Effectivities.

The **string_notation** property from the **effs_Part_BOM_expression** ItemType appears in:

- The **Manage Effectivity Expressions** split pane on the **BOM Structure** tab of the **Part** Item view when the **View Effectivity** context menu item is accessed.
- The **Effectivity** tab of the **Part BOM** Relationship Item view.

The **eff_expression_string_notation** property on the **Part BOM** ItemType is displayed in:

- The **BOM Structure** tab of the **Part** Item view in the **Effectivity** column.

6.10.1 Customizing Effectivity String Notation on Effectivity Expression

Upon its creation and update, an Effectivity Expression gets a new Effectivity String Notation. This behavior is the job of the **effs_expStringNotationConversion** server-side Method executed on the **onBeforeAdd** and **onBeforeUpdate** events of the **effs_Part_BOM_expression** Items. This Method converts an Effectivity Condition to an Effectivity String Notation and stores the output to the **string_notation** Item property.

Customization of the Effectivity String Notations requires modification of the discussed Method as follows:

1. Define a *new class* to represent a new Effectivity String Notation converter and to provide a String Notation value for a given Effectivity Expression object representation. This *class* should get objects of Effectivity Variables and Effectivity Expression, convert them into Effectivity String Notation, and return this Notation. The *class* must implement the `Aras.Server.Core.Configurator.IStringNotationConverter` interface, as included in the Appendix.

Note: A new class for a custom Effectivity String Notation converter is required because the `Aras.Server.Core.Configurator.ExpressionToStringNotationConverter` class with the default Effectivity String Notation converter is compiled. A customization example is provided in the Appendix.

2. In the `CreateNewStringNotationConverter` local method definition, replace the `Aras.Server.Core.Configurator.ExpressionToStringNotationConverter` class with your custom class in the return statement. Do not change the `variableContainer` argument.

```
internal virtual Aras.Server.Core.Configurator.IStringNotationConverter
CreateNewStringNotationConverter(
IEnumerable<Aras.Server.Core.Configurator.Variable> variableContainer)

{

return new
Aras.Server.Core.Configurator.ExpressionToStringNotationConverter(

variableContainer);
```

```
}
```

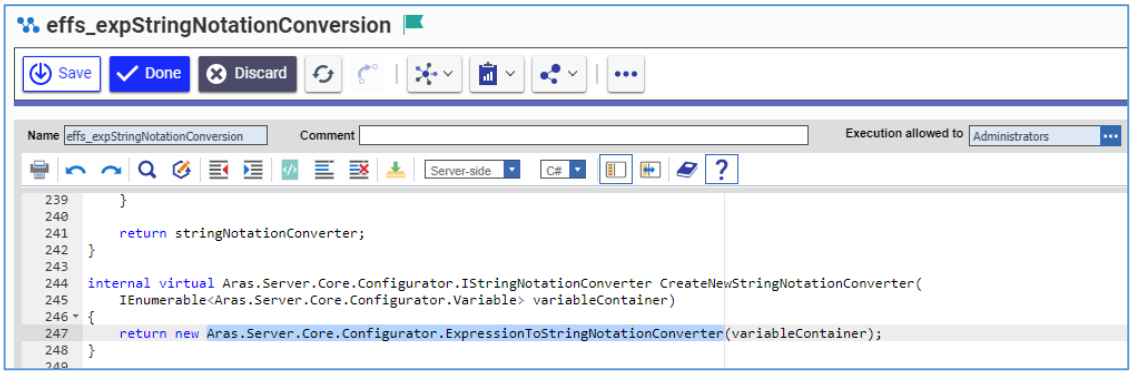

Figure 57.

6.10.2 Customizing Unified Effectivity String Notation

The **Part BOM** ItemType is the **Effectivity Scope ItemType** of the **Aras Part BOM Scope**. An ItemType used as the **Effectivity Scope ItemType** can have a unified Effectivity String Notation joined from its related single Effectivity Expressions. To provide this feature, the **Part BOM** ItemType uses the:

- **eff_expression_string_notation** federated property.
- **effs_getExpressionStringNotation** server-side Method executed on the **onAfterGet** event.

Customization of the unified Effectivity String Notations requires modification of this Method, where the `JoinExpressionStringNotation` local method definition is changed according to the desired uniting template.

Note: A customization example is provided in the Appendix.

The local method produces a single value for the **eff_expression_string_notation** property. It combines related Effectivity Expressions using a conjunction.

The following is the default implementation of `JoinExpressionStringNotation` where parentheses enclose individual Effectivity String Notations and commas join these Notations.

```
internal virtual string JoinExpressionStringNotation(IEnumerable<string>
expressionStringNotations)

{
  if
  (!string.IsNullOrEmpty(expressionStringNotations.ElementAtOrDefault(1)))
  {
     expressionStringNotations = expressionStringNotations.Select(
     expressionStringNotation => "(" + expressionStringNotation + ")");
  }

  return string.Join(", ", expressionStringNotations);
}
```

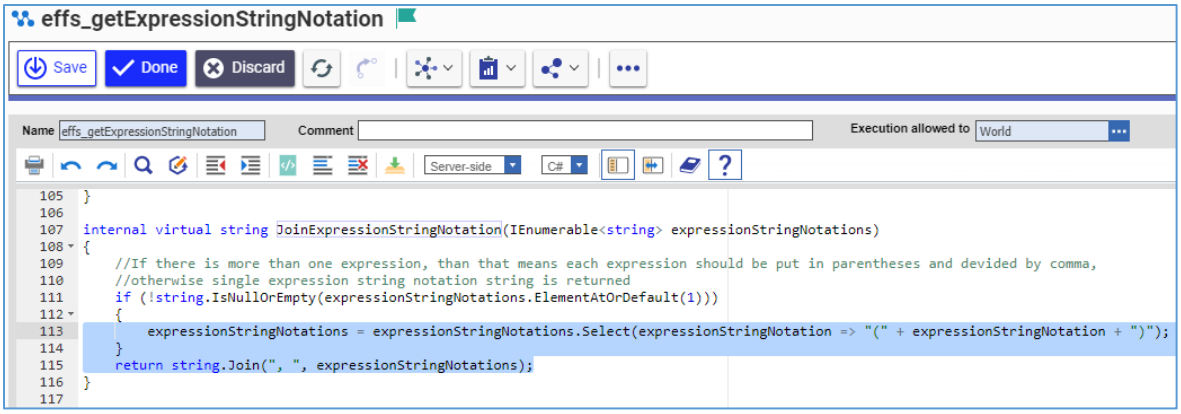

Figure 58.

7 Configuring BOM Reports

To access or create a Report, select → **Administration** → **Reports** from the TOC.

7.1 Understanding BOM Report Implementation

The high-level flow of the BOM Report implementation is as follows:

- When a user executes a BOM Report, its corresponding client method is called.
- The server method **PE_ExecuteQdReportMethod** is run through the client method.
- **PE_ExecuteQdReportMethod** calls the Query Definition (QD) for the selected BOM Report.
- The QD provides queries to retrieve report-specific data from the database.
- **PE_ExecuteQdReportMethod** parses results from the QD execution, and performs report specific logic, such as quantity roll-up. The resulting data in XML format includes level property and is in its final order so it can be transformed into HTML in a hierarchical view.
- The report stylesheet parses data in XML format into HTML using XSLT.

Table 2: Client Methods and Query Definitions for BOM Reports

BOM Report	Client Method	Query Definition
BOM Costing Report	**PE_executeCostingRepAndApplyXslt**	**PE_costingReport**
Multilevel BOM Report	**PE_execMultilevelRepAndApplyXslt**	**PE_multilevelReport**
BOM Quantity Rollup Report	**PE_execQuantityRepAndApplyXslt**	**PE_quantityReport**

The standard BOM Reports can be customized to add, remove, replace, and reorder columns. To customize a standard BOM report, configure its:

1. Query Definition (QD) to retrieve necessary properties.
2. Stylesheet to display these properties.

Note: Customization examples are provided in the Appendix.

7.2 Setting the Number of Rows in BOM Reports

By default, the BOM Reports are limited to 1000000 rows. Opening a larger BOM structure causes an error.

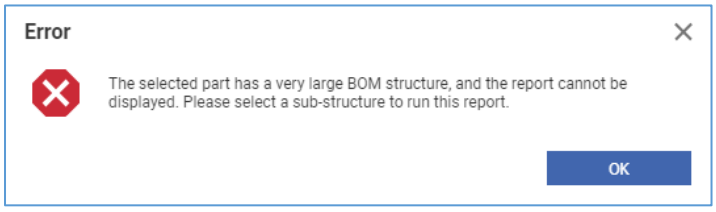

Figure 59.

If your company manages smaller BOM structures, you can limit BOM Reports to a specific number of rows using the following procedure:

1. Search for the **PE_ExecuteQdReportMethod** Method and open its item view for editing.
2. Go to the **PE_ExecuteQdReportMethod** Method editor.
3. Change `1000000` to *`the specific number`* at all occurrences of the following `if` statement:

```
if (_partsTree.Count >= 1000000)
{
    throw new
    Aras.Server.Core.InnovatorServerException(errorLookup.Lookup("PE_Repo
    rtIsVeryLarge"));
}
```

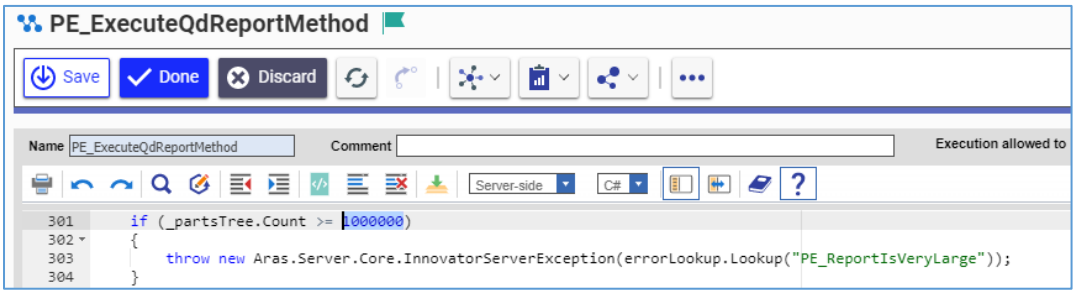

Figure 60.

4. Click **Done** on the **PE_ExecuteQdReportMethod** Method toolbar and close its item view.

8 Appendix

8.1 Aras.Server.Core.Configurator.IStringNotationConverter Interface Definition

The `Aras.Server.Core.Configurator.IStringNotationConverter` interface requires its implementors to support the `ConvertExpressionToStringNotation` method. This method provides a String Notation value to a given Effectivity Expression object representation.

The `Aras.Server.Core.Configurator.IStringNotationConverter` interface definition is:

```
namespace Aras.Server.Core.Configurator
{
   public interface IStringNotationConverter
   {
      string ConvertExpressionToStringNotation(ExpressionBase expression);
   }
}
```

8.2 Examples of Custom Effectivity String Notation

This section provides examples of customizing the Effectivity String Notation. These simple examples supplement the concepts detailed in sections *7.10.1 Customizing Effectivity String Notation on Effectivity Expression* and *7.10.2 Customizing Unified Effectivity String Notation*.

8.2.1 Custom Effectivity String Notation on Effectivity Expression

The default Effectivity String Notations Converter uses a conjunction word for each logical operator:

- **AND** for logical conjunction
- **OR** for logical disjunction
- **NOT** for logical complement

For example:

((Factory = Munich **AND** Production Date >= 01/01/2020) **OR**
(Factory = Detroit **AND** Production Date >= 06/01/2020)) **AND**
NOT (Body Type = Hatchback)

The customized String Notation will use the following characters instead of the conjunction words:

- **&&** for logical conjunction
- **||** for logical disjunction
- **!** for logical complement

For example:

((Factory = Munich **&&** Production Date >= 01/01/2020) **||**
(Factory = Detroit **&&** Production Date >= 06/01/2020)) **&&**
!(Body Type = Hatchback)

8.2.1.1 ExpressionToShortStringNotationConverter Class Definition

To achieve this customization goal, a new `ExpressionToShortStringNotationConverter` class is defined representing the new Converter:

```
internal class ExpressionToShortStringNotationConverter :
Aras.Server.Core.Configurator.IStringNotationConverter
{
   private readonly IEnumerable<Aras.Server.Core.Configurator.Variable>
_variablesContainer;

   private static readonly Dictionary<Type, Aras.Server.Core.Configurator.DataType>
typeTranslator = new Dictionary<Type, Aras.Server.Core.Configurator.DataType>
   {
      { typeof(string), Aras.Server.Core.Configurator.DataType.String },
      { typeof(int), Aras.Server.Core.Configurator.DataType.Int },
      { typeof(DateTime), Aras.Server.Core.Configurator.DataType.DateTime }
   };

   /// <summary>
   /// Initializes a new instance of the ExpressionToShortStringNotationConverter
class with the defined variables container
   /// </summary>
   /// <param name="variablesContainer">Represents a list of
"Aras.Server.Core.Configurator.Variable" objects, which will be used to get human-
readable identifiers of Variables and/or Named-constants</param>
   public
CustomExpressionToStringNotationConverter(IEnumerable<Aras.Server.Core.Configurator.Va
riable> variablesContainer)
   {
      if (variablesContainer == null)
      {
         throw new ArgumentNullException("variablesContainer");
      }

      this._variablesContainer = variablesContainer;
   }

   /// <summary>
   /// Converts an incoming "Aras.Server.Core.Configurator.ExpressionBase" Effectivity
Expression object representation to a string notation and returns it.
   /// </summary>
   /// <param name="expression">An Effectivity Expression object
representation</param>
   /// <returns>A effectivity string notation value</returns>
   public string
ConvertExpressionToStringNotation(Aras.Server.Core.Configurator.ExpressionBase
expression)
   {
      if (expression == null)
      {
         throw new ArgumentNullException("expression");
      }

      return TranslateToStringNotationImplementation(expression, null);
   }

   private string
TranslateToStringNotationImplementation(Aras.Server.Core.Configurator.ExpressionBase
expression, Aras.Server.Core.Configurator.ExpressionBase parentExpression)
   {
      if (expression.ExpressionType ==
Aras.Server.Core.Configurator.ExpressionType.Term)
```

```
        {
            Aras.Server.Core.Configurator.ExpressionTerm expressionTerm = expression as
Aras.Server.Core.Configurator.ExpressionTerm;

            string variableId = expressionTerm.OperandLeft.GetId();
            Aras.Server.Core.Configurator.Variable variable =
_variablesContainer.FirstOrDefault(var => var.Id == variableId);
            string variableName = variable?.Name ?? variableId;

            string assignedVariableValue;
            Aras.Server.Core.Configurator.ITermOperand expressionTermOperandRight =
expressionTerm.OperandRight;
            if (expressionTermOperandRight.OperandType ==
Aras.Server.Core.Configurator.TermOperandType.NamedConstant)
            {
                string namedConstantId = expressionTermOperandRight.GetId();
                assignedVariableValue =
variable?.Enum?.FindNamedConstantById(namedConstantId)?.Name ?? namedConstantId;
            }
            else if (expressionTermOperandRight.OperandType ==
Aras.Server.Core.Configurator.TermOperandType.Constant)
            {
                object constantValue = expressionTermOperandRight.GetValue();
                Type constantType = constantValue.GetType();

                Aras.Server.Core.Configurator.DataType constantDataType;
                if (!typeTranslator.TryGetValue(constantType, out constantDataType))
                {
                    throw new
NotSupportedException(FormattableString.Invariant($"'{constantType.FullName}' constant
data type is not supported as the right operand type for string notation
conversion"));
                }
                //'null' as the first parameter of the 'ToLocalString()' method call
indicates the incoming DateTime object shouldn't be adjusted to any time zone, because
it's assumed 'datetime' always is returned in neutral format with UTC time zone
                assignedVariableValue = constantDataType !=
Aras.Server.Core.Configurator.DataType.DateTime ? Convert.ToString(constantValue,
CultureInfo.InvariantCulture) : Aras.I18NUtils.DateTimeConverter.ToLocalString(null,
(DateTime)constantValue);
            }
            else
            {
                throw new
NotSupportedException(FormattableString.Invariant($"{expressionTerm.OperandRight.Opera
ndType} is not supported as the right operand type for string notation conversion"));
            }

            return string.Format(CultureInfo.InvariantCulture, "{0} {1} {2}",
AddSquareBracketsIfNeed(variableName),
GetAlgebraicTermOperatorNotation(expressionTerm.Operator),
AddSquareBracketsIfNeed(assignedVariableValue));
        }

        string expressionStringNotation = string.Empty;
        string expressionOperatorStrNotation =
GetAlgebraicNotationByOperator(expression.ExpressionType);
        //we assume at-least-one, at-most-one, exactly-one expressions consists of
terms(<EQ><variable id="" /><named-constant id="" /></EQ>) only here
        if (expression.ExpressionType ==
Aras.Server.Core.Configurator.ExpressionType.AtLeastOne ||
```

```
          expression.ExpressionType ==
Aras.Server.Core.Configurator.ExpressionType.AtMostOne ||
          expression.ExpressionType ==
Aras.Server.Core.Configurator.ExpressionType.ExactlyOne)
       {
          IEnumerable<string> innerTermsStringNotation =
expression.InnerExpressions.Select(expr =>
TranslateToStringNotationImplementation(expr, expression));
          return string.Format(CultureInfo.InvariantCulture,
expressionOperatorStrNotation, string.Join(" | ", innerTermsStringNotation));
       }

       foreach (Aras.Server.Core.Configurator.ExpressionBase innerExpression in
expression.InnerExpressions)
       {
          string innerExpressionStrNotation =
TranslateToStringNotationImplementation(innerExpression, expression);
          if (!string.IsNullOrEmpty(innerExpressionStrNotation))
          {
             if (expression.ExpressionType ==
Aras.Server.Core.Configurator.ExpressionType.NOT)
             {
                expressionStringNotation +=
string.Format(CultureInfo.InvariantCulture, expressionOperatorStrNotation,
innerExpressionStrNotation);
             }
             else if (expression.ExpressionType ==
Aras.Server.Core.Configurator.ExpressionType.IMPLICATION)
             {
                string[] ifThen = expressionOperatorStrNotation.Split(' ');
                bool isStringNotationEmpty =
string.IsNullOrEmpty(expressionStringNotation);

                // implicationTemplate describes "IF {expr}" or "THEN {expr}". In
case "IF", whitespace after "{expr}" is required -"IF {expr} "
                string implicationTemplate = isStringNotationEmpty ? "{0} {1} " :
"{0} {1}";
                expressionStringNotation +=
string.Format(CultureInfo.InvariantCulture, implicationTemplate,
ifThen[isStringNotationEmpty ? 0 : 1], innerExpressionStrNotation);
             }
             else
             {
                expressionStringNotation +=
!string.IsNullOrEmpty(expressionStringNotation) ? expressionOperatorStrNotation +
innerExpressionStrNotation : innerExpressionStrNotation;
             }
          }
       }

       return parentExpression != null && parentExpression.InnerExpressions.Count > 1
&&
          parentExpression.ExpressionType !=
Aras.Server.Core.Configurator.ExpressionType.IMPLICATION && expression.ExpressionType
!= Aras.Server.Core.Configurator.ExpressionType.NOT ?
          "(" + expressionStringNotation + ")" : expressionStringNotation;
    }

    private static string AddSquareBracketsIfNeed(string name)
    {
       return !string.IsNullOrEmpty(name) &&
!System.Text.RegularExpressions.Regex.IsMatch(name, @"^[a-zA-Z0-9]+$") ?
```

```
            string.Format(CultureInfo.InvariantCulture, "[{0}]", name) : name;
    }

    private static string
GetAlgebraicNotationByOperator(Aras.Server.Core.Configurator.ExpressionType
boolOperator)
    {
        string notation = string.Empty;
        switch (boolOperator)
        {
            case Aras.Server.Core.Configurator.ExpressionType.AND:
                notation = " && ";
                break;
            case Aras.Server.Core.Configurator.ExpressionType.OR:
                notation = " || ";
                break;
            case Aras.Server.Core.Configurator.ExpressionType.NOT:
                notation = "!({0})";
                break;
            case Aras.Server.Core.Configurator.ExpressionType.IMPLICATION:
                notation = "IF THEN";
                break;
            case Aras.Server.Core.Configurator.ExpressionType.AtLeastOne:
                notation = "AT-LEAST-ONE({0})";
                break;
            case Aras.Server.Core.Configurator.ExpressionType.AtMostOne:
                notation = "AT-MOST-ONE({0})";
                break;
            case Aras.Server.Core.Configurator.ExpressionType.ExactlyOne:
                notation = "EXACTLY-ONE({0})";
                break;
            default:
                throw new NotSupportedException(boolOperator + " type is not supported
for string notation conversion");
        }

        return notation;
    }

    private static string
GetAlgebraicTermOperatorNotation(Aras.Server.Core.Configurator.TermOperator
termOperator)
    {
        string algebraicTermOperatorNotation;
        switch (termOperator)
        {
            case Aras.Server.Core.Configurator.TermOperator.Equal:
                algebraicTermOperatorNotation = "=";
                break;
            case Aras.Server.Core.Configurator.TermOperator.GreaterThanOrEqual:
                algebraicTermOperatorNotation = ">=";
                break;
            case Aras.Server.Core.Configurator.TermOperator.LessThanOrEqual:
                algebraicTermOperatorNotation = "<=";
                break;
            default:
                throw new
NotSupportedException(FormattableString.Invariant($"'{termOperator}' term operator is
not supported for string notation conversion"));
        }

        return algebraicTermOperatorNotation;
```

```
    }
}
```

8.2.2 Custom Unified Effectivity String Notation

By default, a comma (,) is used to combine multiple Effectivities on a single Relationship. A customized String Notation uses the OR conjunction word instead of the comma.

Table 3: A customized String Notation example

Effective Item	Single Effectivity String Notations	Unified Effectivity String Notation	
		Default	Customized
Engine	Factory = Detroit	(Factory = Detroit), (Factory = Munich)	(Factory = Detroit) OR (Factory = Munich)
	Factory = Munich		

8.2.2.1 JoinExpressionStringNotation Method Definition

To achieve this customization goal, change the definition of the `JoinExpressionStringNotation` internal method of the **effs_getExpressionStringNotation** Method as follows:

```
internal virtual string JoinExpressionStringNotation(IEnumerable<string>
expressionStringNotations)
{
    if (!string.IsNullOrEmpty(expressionStringNotations.ElementAtOrDefault(1)))
    {
        expressionStringNotations = expressionStringNotations.Select(
        expressionStringNotation => "(" + expressionStringNotation + ")");
    }
    return string.Join(" OR ", expressionStringNotations);
}
```

8.3 Examples of Custom BOM Reports

This section provides examples of customizing BOM Reports. These simple examples supplement the concept described in section *8 Configuring BOM Reports*.

8.3.1 Adding a New Column Using an Item Property

In this example, the Multilevel BOM Report is customized. Using the following procedure, the **Unit** column is added between the **Quantity** and **AML Status** columns to display the **Unit** property of **Part** Items:

1. Search for the **PE_multilevelReport** Query Definition (QD) and open its item view for editing.
2. Click **Show Editor** on the **PE_multilevelReport** QD sidebar.
3. Click the **Properties** button on the top **RootPart** row. The **Properties** dialog box appears.

4. Add the **Unit** property to the **Properties** dialog box and click **Save**.

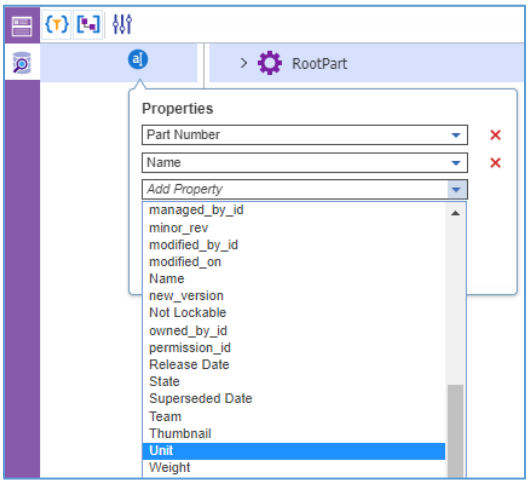

Figure 61.

5. Repeat steps 3-4 on the **Part** row representing the Part Query Item.

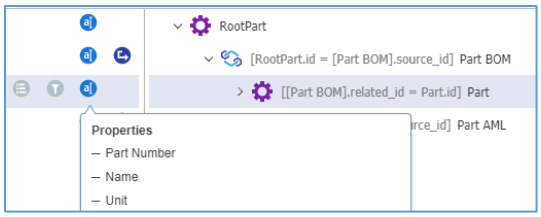

Figure 62.

Note: Both **Root** and **Part** Query Items should have the same Properties in a QD. Otherwise, Aras Innovator will return an error. This requirement does not apply to other Query Items of the QD.

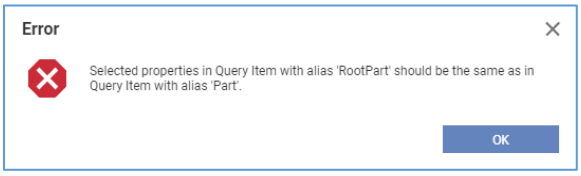

Figure 63.

6. Click **Save, Unlock & Close** on the **PE_multilevelReport** QD toolbar.
7. Search for the **Multilevel BOM Report** and open its Item view for editing.
8. Go to the **Stylesheet** editor tab.

9. Add the `<th>Unit</th>` table header cell element after `<th>Quantity</th>`.

```
<table border="0" cellspacing="0" cellpadding="0">
    <tr>
        <th colspan="{$MaxDepth + 1}">BOM Level</th>
        <th>Part Number</th>
        <th>Name</th>
        <th>Quantity</th>
        <th>Unit</th>
        <th>AML Status</th>
        <th>Manufacturer</th>
        <th>Manufacturer Part</th>
    </tr>
    <xsl:call-template name="rootItems"/>
</table>
```

Figure 64.

10. Add a table data cell element for the `unit` property values after the one for `quantity`.

```
<td rowspan="{$rowCount}" width="120px">
      <xsl:value-of select="unit"/>
</td>
```

```
<td rowspan="{$rowCount}" width="80px" align="center">
    <xsl:choose>
        <xsl:when test="depth=0">1</xsl:when>
        <xsl:when test="depth!=0 and (quantity)="">1</xsl:when>
        <xsl:otherwise>
            <xsl:value-of select="quantity"/>
        </xsl:otherwise>
    </xsl:choose>
</td>
<td rowspan="{$rowCount}" width="120px">
    <xsl:value-of select="unit"/>
</td>
<td width="100px">
    <xsl:value-of select="Relationships/Item[@alias='Part AML'][1]/state"/>
</td>
```

Figure 65.

11. Click **Apply** on the **Stylesheet** editor tab.
12. Click **Done** on the **Multilevel BOM Report** toolbar. The **Unit** column is added to the Multilevel BOM Report.

Bill of Materials Report — Generated on: 7/5/2019, 4:23:46 PM

BOM Level	Part Number	Name	Quantity	Unit	AML Status	Manufacturer	Manufacturer Part
0	Parent Part	Parent Part	1	EA			
1	Child Part A	Child Part A	1	EA			
2	Child Part A.1	Child Part A.1	1	EA			
2	Child Part A.2	Child Part A.2	1	EA			
1	Child Part B	Child Part B	1	EA			
2	Child Part B.1	Child Part B.1	1	EA			
2	Child Part B.2	Child Part B.2	1	EA			

Figure 66.

8.3.2 Adding a New Column Using a Relationship Property

Note: This example can be used as a guide for adding any relationship property. For the **Sequence** relationship property, prefer the configuration as described in the next section.

In this example, the BOM Costing Report is customized. Using the following procedure, the **Reference Designator** column is added as the last column to display the **Reference Designator** property of the **Part BOM** Relationship ItemType:

1. Search for the **PE_costingReport** Query Definition (QD) and open its item view for editing.
2. Click **Show Editor** on the **PE_costingReport** QD sidebar.

3. Click the **Properties** button on the **Part BOM** row. The **Properties** dialog box appears.
4. Add the **Reference Designator** property to the **Properties** dialog box and click **Save**.

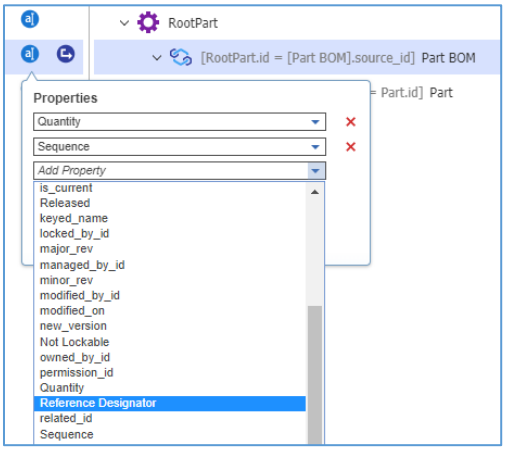

Figure 67.

5. Click **Save, Unlock & Close** on the **PE_costingReport** QD toolbar.
6. Search for the **BOM Costing Report** and open its Item view for editing.
7. Go to the **Stylesheet** editor tab.
8. Add the `<th> Reference Designator</th>` table header cell element after `<th>Cost Basis</th>`.

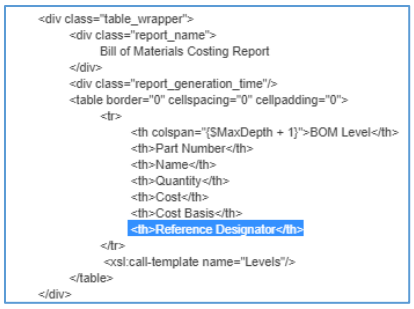

Figure 68.

9. Add a table data cell element for the `reference_designator` property values after the one for `cost_basis`.

```
<td>
        <xsl:value-of select="Relationships/Item[@alias='Part BOM' and
        @info='parent']/reference_designator"/>
</td>
```

Note: When customizing the Multilevel BOM Report with a Relationship ItemType property, this property table data tag should have the `rowspan` attribute as follows:

```
<td rowspan="{$rowCount}" width="120px">
     <xsl:value-of select="Relationships/Item[@alias='Part BOM' and
     @info='parent']/property_name"/>
</td>
```

```
                    <td>
                        <xsl:value-of select="cost_basis"/>
                        <xsl:if test="cost_basis='' or not(cost_basis)">
                          <xsl:text> </xsl:text>
                        </xsl:if>
                    </td>
                    <td>
                        <xsl:value-of select="Relationships/Item[@alias='Part BOM' and @info='parent']/reference_designator"/>
                    </td>
                </tr>
            </xsl:template>
</xsl:stylesheet>
```

Figure 69.

10. Click **Apply** on the **Stylesheet** editor tab.
11. Click **Done** on the **BOM Costing Report** toolbar. The **Reference Designator** column is added to the BOM Costing Report.

Bill of Materials Costing Report — Generated on: 10/15/2019, 12:50:33 PM

BOM Level	Part Number	Name	Quantity	Cost	Cost Basis	Reference Designator
0	Parent Part	Parent Part	1	16.0000	Calculated	
1	Child Part A	Child Part A	1	10.0000	Calculated	RD P-A
2	Child Part A.1	Child Part A.1	1	7.0000	Actual	RD A-A1
2	Child Part A.2	Child Part A.2	1	3.0000	Actual	RD A-A2
1	Child Part B	Child Part B	1	6.0000	Calculated	RD P-B
2	Child Part B.1	Child Part B.1	1	2.0000	Actual	RD B-B1
2	Child Part B.2	Child Part B.2	1	4.0000	Actual	RD B-B2

Figure 70.

8.3.3 Adding the Sequence Column

In this example, the BOM Costing Report is customized. Using the following procedure, the **Sequence** column is added as the last column to show the **Sequence** property of the **Part BOM** Relationship ItemType:

1. Search for the **BOM Costing Report** and open its Item view for editing.
2. Go to the **Stylesheet** editor tab.
3. Add the `<th> Sequence</th>` table header cell element after `<th>Cost Basis</th>`.

```
<div class="table_wrapper">
    <div class="report_name">
        Bill of Materials Costing Report
    </div>
    <div class="report_generation_time"/>
    <table border="0" cellspacing="0" cellpadding="0">
        <tr>
            <th colspan="{$MaxDepth + 1}">BOM Level</th>
            <th>Part Number</th>
            <th>Name</th>
            <th>Quantity</th>
            <th>Cost</th>
            <th>Cost Basis</th>
            <th>Sequence</th>
        </tr>
        <xsl:call-template name="Levels"/>
    </table>
</div>
```

Figure 71.

4. Add a table data cell element for the `sort_order` property values after the one for `cost_basis`.

```
<td>
        <xsl:value-of select="sort_order"/>
```

```
</td>
```

Note: When customizing the Multilevel BOM Report with the Sequence property, this property table data tag should have the `rowspan` attribute as follows:

```
<td rowspan="{$rowCount}" width="120px">
     <xsl:value-of select="sort_oder"/>
</td>
```

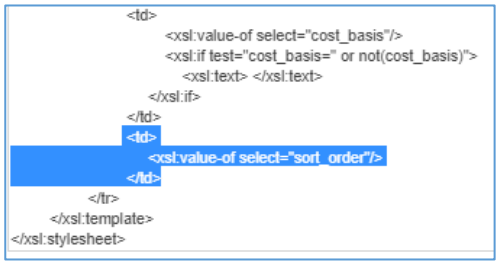

Figure 72.

5. Click **Apply** on the **Stylesheet** editor tab.
6. Click **Done** on the **BOM Costing Report** toolbar. The **Sequence** column is added to the BOM Costing Report.

Bill of Materials Costing Report — Generated on: 10/15/2019, 1:33:40 PM

BOM Level	Part Number	Name	Quantity	Cost	Cost Basis	Sequence
0	Parent Part	Parent Part	1	16.0000	Calculated	
1	Child Part A	Child Part A	1	10.0000	Calculated	1
2	Child Part A.1	Child Part A.1	1	7.0000	Actual	1
2	Child Part A.2	Child Part A.2	1	3.0000	Actual	2
1	Child Part B	Child Part B	1	6.0000	Calculated	2
2	Child Part B.1	Child Part B.1	1	2.0000	Actual	1
2	Child Part B.2	Child Part B.2	1	4.0000	Actual	2

Figure 73.